The Zak Transform

(Direct from definition, Not through ISFFT and Heisenberg Transform)

Sampled, finite−pulse, FFT−accelerated
direct Zak implementation is practical

DEFINITION and COMPUTATIONAL RESOURCE REQUIREMENT
AND EASE OF SWITCHING

The Zak Transform (Finite-frame pulse-shaped)

$$Y[m,n] = \sum_{q=0}^{N-1} \int r(t)\, g_{\mathrm{rx}}^{*}(t - mT_s - qT)\, e^{-j2\pi nq/N} dt.$$

Received Waveform $r(t)$

be the received continuous-time OTFS waveform.

Define:

T_s = delay sampling interval

M = number of delay bins

$T = MT_s$ = OTFS delay period

N = number of Doppler observation periods

$m = 0, \ldots, M-1$ = delay-bin index

$q = 0, \ldots, N-1$ = period index

$n = 0, \ldots, N-1$ = Doppler-bin index

$g_{\mathrm{rx}}(t)$ = receive pulse

OTFS Frame time $T_{frame} = NT = NMT_s$

When $g_{\mathrm{rx}}^{*}(t) = \delta(t)$

Then

$$g_{\mathrm{rx}}^{*}(t - mT_s - qT) = \delta(t - mT_s - qT)$$

Substitute:

$$Y[m,n] = \sum_{q=0}^{N-1} \int r(t)\delta(t - mT_s - qT)e^{-j2\pi nq/N} dt$$

Using sifting property:

$$\int r(t)\delta(t-a)dt = r(a)$$

gives:

$$Y[m,n] = \sum_{q=0}^{N-1} r(mT_s + qT)e^{-j2\pi nq/N}$$

Normally seen in books

The Zak Transform with pulse-shaping

Now sample:

$$r[l] = r(lT_s)$$

with

$$T = MT_s$$

Then:

$$mT_s + qT = (m + qM)T_s$$

thus:

$$\boxed{Y[m,n] = \sum_{q=0}^{N-1} r[m + qM] e^{-j2\pi nq/N}}$$

With a general receive pulse $g_{\text{rx}}(t)$, you **cannot collapse the integral** to $r(a)$. So the DD-domain value remains a projection:

$$\boxed{Y[m,n] = \sum_{q=0}^{N-1} \int_{-\infty}^{\infty} r(t)\, g_{\text{rx}}^*(t - mT_s - qT)\, e^{-j2\pi nq/N}\, dt}$$

Define the intermediate delay-period projection:

$$\boxed{u_m[q] = \int_{-\infty}^{\infty} r(t)\, g_{\text{rx}}^*(t - mT_s - qT)\, dt}$$

Then

$$\boxed{Y[m,n] = \sum_{q=0}^{N-1} u_m[q] e^{-j2\pi nq/N}}$$

So with general $g_{\text{rx}}(t)$, $Y[m,n]$ is not a point sample. It is a **matched-filter/projection output followed by Doppler accumulation**.

r(t)→receive-pulse projection → um[q]→DZT over q → Y[m,n]

$$\int_{-\infty}^{\infty} |g(t)|^2\, dt = 1 \qquad \boxed{\sum_a |g[a]|^2 = 1} \qquad \boxed{g[a] \leftarrow \frac{g[a]}{\sqrt{\sum_a |g[a]|^2}}}$$

The Zak Transform sampled form

Let the ADC sample the received waveform at

$$t = aT_{\text{samp}}, a \in \mathbb{Z},$$

and define

$$r[a] \equiv r\left(aT_{\text{samp}}\right), g_{\text{rx}}[\ell] \equiv g_{\text{rx}}\left(\ell T_{\text{samp}}\right).$$

Also define

$$D \equiv \frac{T_s}{T_{\text{samp}}},$$

and since

$$T = MT_s,$$

define

$$P \equiv \frac{T}{T_{\text{samp}}} = \frac{MT_s}{T_{\text{samp}}} = MD.$$

Therefore, the continuous pulse centre

$$mT_s + qT$$

corresponds to the ADC-sample index

$$\boxed{\frac{mT_s + qT}{T_{\text{samp}}} = Dm + Pq = Dm + DMq.}$$

1. Continuous receive projection

The intermediate projection is

$$u_m[q] = \int_{-\infty}^{\infty} r\,(t)\, g_{\text{rx}}^*(t - mT_s - qT)\, dt.$$

Approximating the integral by ADC-rate samples gives

$$u_m[q] \approx T_{\text{samp}} \sum_{a=-\infty}^{\infty} r\,[a]\, g_{\text{rx}}^*[a - Dm - Pq]$$

Equivalently, using $P = MD$,

$$\boldsymbol{u_m[q] \approx T_{\text{samp}} \sum_{a=-\infty}^{\infty} r\,[a]\, g_{\text{rx}}^*[a - Dm - DMq]}$$

This is the sampled matched-filter/projection operation.

The Zak Transform sampled form

2. More convenient FIR form

Define the relative pulse-sample index

$$\ell = a - Dm - Pq.$$

Therefore,

$$a = Dm + Pq + \ell.$$

Substituting gives

$$u_m[q] \approx T_{\text{samp}} \sum_{\ell=-\infty}^{\infty} r\,[Dm + Pq + \ell]\, g_{\text{rx}}^*[\ell]$$

If only L_g pulse samples are retained and L_g is odd, define

$$L_- = L_+ = \frac{L_g - 1}{2}.$$

Then

$$u_m[q] \approx T_{\text{samp}} \sum_{\ell=-\frac{L_g-1}{2}}^{\frac{L_g-1}{2}} r\,[Dm + Pq + \ell]\, g_{\text{rx}}^*[\ell]$$

This is the most convenient hardware form: for every (m, q), read L_g received samples centred around

$$a_{m,q} = Dm + Pq.$$

3. Sampled Zak computation

After calculating $u_m[q]$, the DD value is

$$Y[m, n] = \sum_{q=0}^{N-1} u_m\,[q] e^{-j2\pi nq/N}$$

For each fixed m, this is an N-point FFT algorithm over q:

$$Y[m, :] = \text{FFT}_N\{u_m[:]\}$$

4. Complete sampled equation

Substituting the sampled projection into the Zak sum gives

$$\boldsymbol{Y[m, n] \approx T_{\text{samp}} \sum_{q=0}^{N-1} \sum_{a=-\infty}^{\infty} r\,[a]\, g_{\text{rx}}^*[a - Dm - Pq]\, e^{-j2\pi nq/N}}.$$

The Zak Transform sampled form

Using the retained L_g-tap receive pulse,

$$\boldsymbol{Y[m,n] \approx T_{\text{samp}} \sum_{q=0}^{N-1} \sum_{\ell=-\frac{L_g-1}{2}}^{\frac{L_g-1}{2}} r\,[Dm+Pq+\ell]\, g_{\text{rx}}^{*}[\ell]\, e^{-j2\pi nq/N}}.$$

With

$$P = MD,$$

this can also be written as

$$Y[m,n] \approx T_{\text{samp}} \sum_{q=0}^{N-1} \sum_{\ell=-\frac{L_g-1}{2}}^{\frac{L_g-1}{2}} r\,[Dm+DMq+\ell]\, g_{\text{rx}}^{*}[\ell]\, e^{-j2\pi nq/N}$$

5. DSP-normalized form

In hardware, the constant T_{samp} can be absorbed into the receive-filter coefficients:

$$\tilde{g}_{\text{rx}}[\ell] \equiv T_{\text{samp}} g_{\text{rx}}[\ell].$$

Then

$$u_m[q] = \sum_{\ell=-\frac{L_g-1}{2}}^{\frac{L_g-1}{2}} r\,[Dm+Pq+\ell]\, \tilde{g}_{\text{rx}}^{*}[\ell]$$

followed by

$$Y[m,n] = \sum_{q=0}^{N-1} u_m\,[q] e^{-j2\pi nq/N}$$

Complete sampled receiver chain

$$\boldsymbol{r[a] \rightarrow \underbrace{u_m[q] = \sum_{\ell} r\,[Dm+Pq+\ell]\, \tilde{g}_{\text{rx}}^{*}[\ell]}_{\text{matched-filter/projection}}}$$

$$\boldsymbol{\rightarrow \underbrace{Y[m,n] = \sum_{q} u_m\,[q] e^{-j2\pi nq/N}}_{N\text{-point FFT algorithm over } q}}.$$

The important address relationship is

$$mT_s + qT \longrightarrow Dm + Pq = Dm + DMq$$

The Zak Transform general sampled form – Computation Load

Starting from the sampled receiver:

$$u_m[q] = \sum_{\ell=-\frac{L_g-1}{2}}^{\frac{L_g-1}{2}} r\,[Dm + Pq + \ell]\,\tilde{g}^*_{\mathrm{rx}}[\ell], P = MD,$$

followed by

$$Y[m, n] = \sum_{q=0}^{N-1} u_m\,[q] e^{-j2\pi nq/N}.$$

The computation has two parts:

1.receive-pulse projection;
2.N-point FFT algorithm over q.

The factor $T_{\mathrm{samp}}$is assumed absorbed into $\tilde{g}_{\mathrm{rx}}[\ell]$.

1. Receive-pulse projection load

For every pair (m, q), the projection contains $L_g$pulse taps. There are

$$MN$$

values of $u_m[q]$per frame.

Therefore, the projection requires

$$C_{\mathrm{proj,mult}} = MNL_g$$

complex-times-pulse-coefficient multiplications per frame.

Accumulating $L_g$terms requires $L_g - 1$complex additions:

$$C_{\mathrm{proj,add}} = MN(L_g - 1)$$

complex additions per frame.

If $g_{\mathrm{rx}}[\ell]$is real, the multiplication is complex-times-real, not a full complex multiplication.

The Zak Transform general sampled form – Computation Load

2. Zak FFT load

For each fixed m,

$$Y[m,:] = \mathrm{FFT}_N\{u_m[:]\}.$$

A radix-2 N-point FFT requires

$$\frac{N}{2}\log_2 N$$

butterflies.

For all M delay rows:

$$\boxed{C_{\mathrm{Zak,FFT}} = M\frac{N}{2}\log_2 N}$$

complex butterflies per frame.

Using the conventional radix-2 count, each butterfly contains approximately:

- one complex twiddle multiplication;
- two complex additions.

Therefore,

$$\boxed{C_{\mathrm{FFT,mult}} \approx M\frac{N}{2}\log_2 N}$$

complex twiddle multiplications per frame, and

$$\boxed{C_{\mathrm{FFT,add}} = MN\log_2 N}$$

complex additions per frame.

This is a conservative count because butterflies containing trivial twiddles such as $1, -1, j, -j$ require fewer multiplications.

3. Total load per frame

The total multiplication count is

$$\boxed{C_{\mathrm{rx,mult}} = MNL_g + M\frac{N}{2}\log_2 N.}$$

The two multiplication types should remain distinguished:

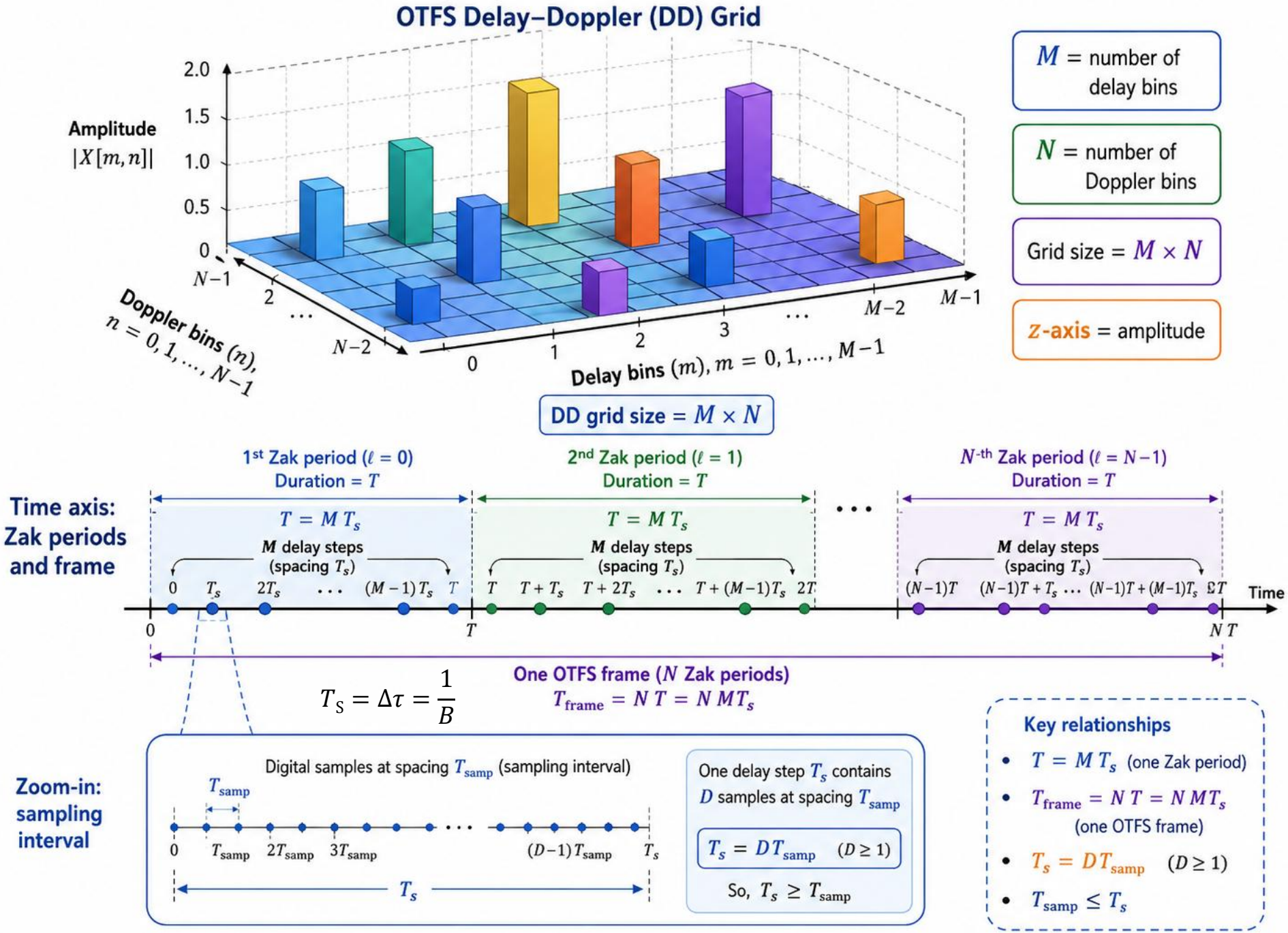
$$\boxed{\boldsymbol{C_{\mathrm{rx}} = MNL_g} \text{ projection multiplications} + \boldsymbol{M\frac{N}{2}\log_2 N} \text{ FFT twiddle multiplications.}}$$

The total complex-addition count is

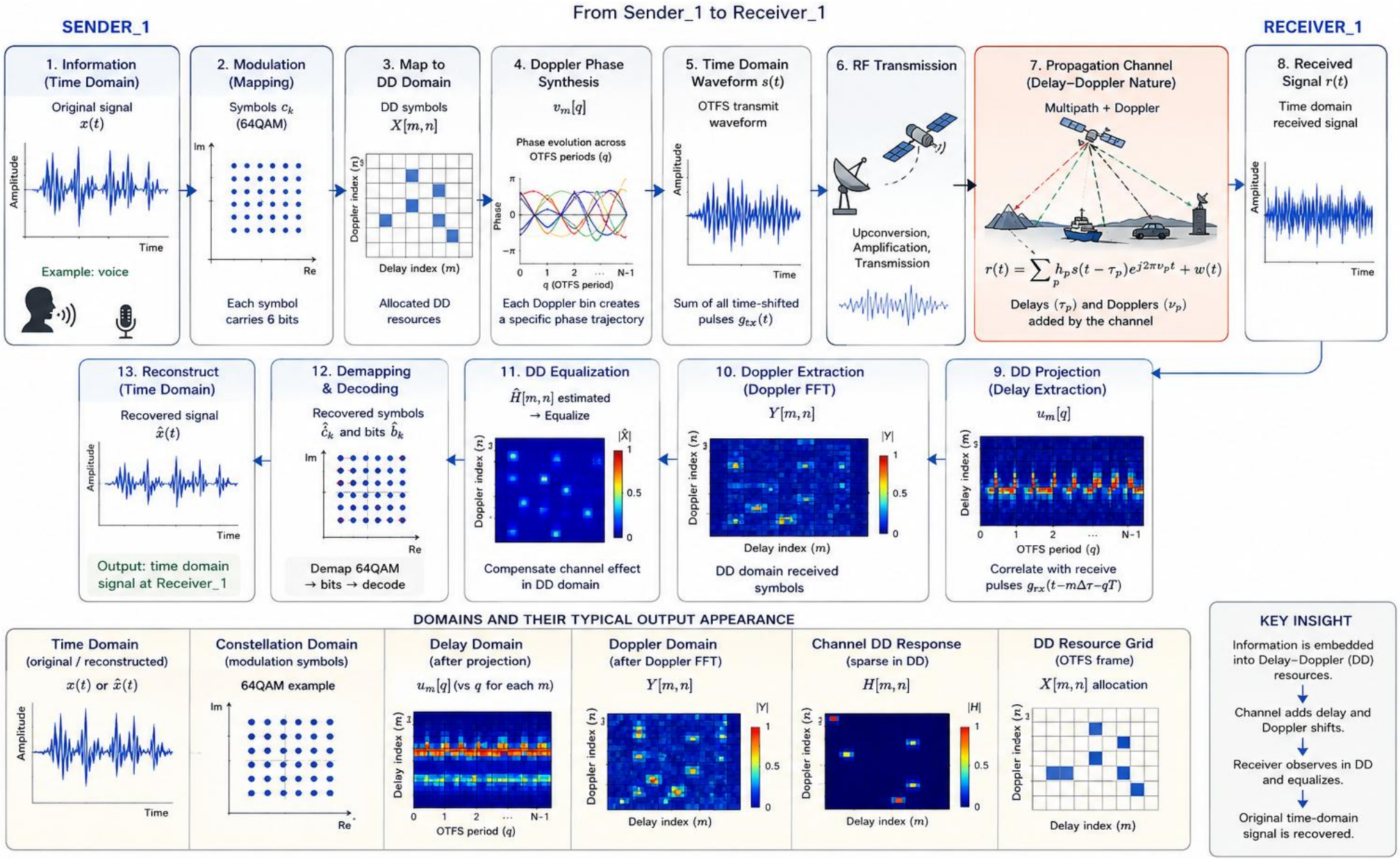
$$C_{\mathrm{rx,add}} = MN(L_g - 1) + MN\log_2 N$$

Equivalently,

$$C_{\mathrm{rx,add}} = MN(L_g - 1 + \log_2 N)$$


OTFS Delay–Doppler (DD) Grid
Amplitude
$|X[m,n]|$
2.0
1.5
1.0
0.5
0
Doppler bins (n), $n = 0, 1, \ldots, N-1$
$N-1$
2
$N-2$
Delay bins (m), $m = 0, 1, \ldots, M-1$
0
1
2
3
$M-2$
$M-1$
M = number of delay bins
N = number of Doppler bins
Grid size = $M \times N$
z-axis = amplitude
DD grid size = $M \times N$
Time axis: Zak periods and frame
1st Zak period ($\ell = 0$)
Duration = T
2nd Zak period ($\ell = 1$)
Duration = T
N-th Zak period ($\ell = N-1$)
Duration = T
$T = M\,T_s$
M delay steps (spacing T_s)
0 T_s $2T_s$ ... $(M-1)\,T_s$ T
T $T+T_s$ $T+2T_s$ ... $T+(M-1)\,T_s$ $2T$
$(N-1)T$ $(N-1)T+T_s$ ... $(N-1)T+(M-1)T_s$ $\mathfrak{L}T$
Time
0
T
$N\,T$
One OTFS frame (N Zak periods)
$T_{\text{frame}} = N\,T = N\,M\,T_s$
$T_S = \Delta\tau = \frac{1}{B}$
Zoom-in: sampling interval
Digital samples at spacing T_{samp} (sampling interval)
T_{samp}
0 T_{samp} $2T_{\text{samp}}$ $3T_{\text{samp}}$ ... $(D-1)\,T_{\text{samp}}$ T_s
T_s
One delay step T_s contains D samples at spacing T_{samp}
$T_s = D\,T_{\text{samp}}$ $(D \geq 1)$
So, $T_s \geq T_{\text{samp}}$
Key relationships
$T = M\,T_s$ (one Zak period)
$T_{\text{frame}} = N\,T = N\,M\,T_s$ (one OTFS frame)
$T_s = D\,T_{\text{samp}}$ $(D \geq 1)$
$T_{\text{samp}} \leq T_s$

What does Zak and IZak mean physically



The Inverse Zak Transform

Invert received coefficients $Y[m, n]$, the output should be ŕ(t)

Start from DD coefficients: $Y[m, n]$.

$$Y[m,n] = \sum_{q=0}^{N-1} u_m[q] e^{-j2\pi nq/N}$$

First recover: $u_m[q]$

$$u_m[q] = \frac{1}{N} \sum_{n=0}^{N-1} Y[m,n] e^{j2\pi nq/N}$$

Then reconstructed ŕ(t) [Also use pulse shaping g(t) for transmission]

$$\acute{r}(t) = \sum_{m=0}^{M-1} \sum_{q=0}^{N-1} \left[\frac{1}{N} \sum_{n} Y[m,n] e^{j2\pi nq/N} \right] g_{\text{tx}}(t - mT_s - qT)$$

For an OTFS transmitter,

$$X[m, n] \rightarrow v_m[q] \rightarrow s(t).$$

Specifically,

$$v_m[q] = \frac{1}{N} \sum_{n=0}^{N-1} X[m, n] e^{j2\pi nq/N},$$

$$s(t) = \sum_{m=0}^{M-1} \sum_{q=0}^{N-1} v_m[q] g_{\text{tx}}(t - mT_s - qT).$$

The Inverse Zak Transform - transmitter side

$$X[m,n] \xrightarrow{\text{IDFT over } n} v_m[q] \xrightarrow{g_{\text{tx}}} s(t),$$

the **generalized inverse Zak transform** is:

$$\boxed{v_m[q] = \frac{1}{N}\sum_{n=0}^{N-1} X[m,n]e^{j2\pi nq/N}}$$

then pulse synthesis:

$$\boxed{s(t) = \sum_{m=0}^{M-1}\sum_{q=0}^{N-1} v_m[q]\, g_{\text{tx}}(t - mT_s - qT)}$$

Substituting $v_m[q]$:

$$\boxed{s(t) = \sum_{m=0}^{M-1}\sum_{q=0}^{N-1}\left[\frac{1}{N}\sum_{n=0}^{N-1} X[m,n]e^{j2\pi nq/N}\right] g_{\text{tx}}(t - mT_s - qT)}$$

Variables:

$X[m, n]$
$=$ transmit DD symbol at delay bin m and Doppler bin n

$v_m[q]$
$=$ time$-$period coefficient after inverse Doppler synthesis

$g_{\text{tx}}(t) =$ unit$-$energy transmit pulse
$m = 0, \dots, M-1$
$n = 0, \dots, N-1$
$q = 0, \dots, N-1$
$T_s =$ delay$-$bin spacing
$T = MT_s =$ OTFS period

So the generalized IZak transmitter is:

$$\boxed{\text{DD symbols} \rightarrow \text{create Doppler phase evolution across } q \rightarrow \text{synthesize waveform using shifted } g_{\text{tx}}(t)}$$

The Inverse Zak Transform - transmitter side (sampled form)

For sampled signal

$$v_m[q] = \frac{1}{N}\sum_{n=0}^{N-1} X[m, n]e^{j2\pi nq/N}$$

$$s[a] = \sum_{m=0}^{M-1}\sum_{q=0}^{N-1} v_m[q]\, g_{\text{tx}}[a-(mD+qP)]$$

or, in one line:

$$\boldsymbol{s[a] = \sum_{m=0}^{M-1}\sum_{q=0}^{N-1}\left[\frac{1}{N}\sum_{n=0}^{N-1} X[m, n]e^{j2\pi nq/N}\right] g_{\text{tx}}[a-(mD+qP)]}.$$

$D = 1$ critical sampling and $D > 1$ oversampled DAC hardware.

From continuous to sampled

$$t = aT_{\text{samp}}, \qquad D = \frac{T_s}{T_{\text{samp}}}, \qquad T = MT_s$$

Delay-grid spacing: $T_s = \frac{1}{B}$

DAC sampling interval:

$$T_{\text{samp}} = \frac{T_s}{D} \;\; so, D = \frac{T_s}{T_{\text{samp}}}$$

The number of DAC-rate samples in one Zak period T.

$$\boldsymbol{T = MT_s}$$

and

$$T_{\text{samp}} = \frac{T_s}{D}.$$

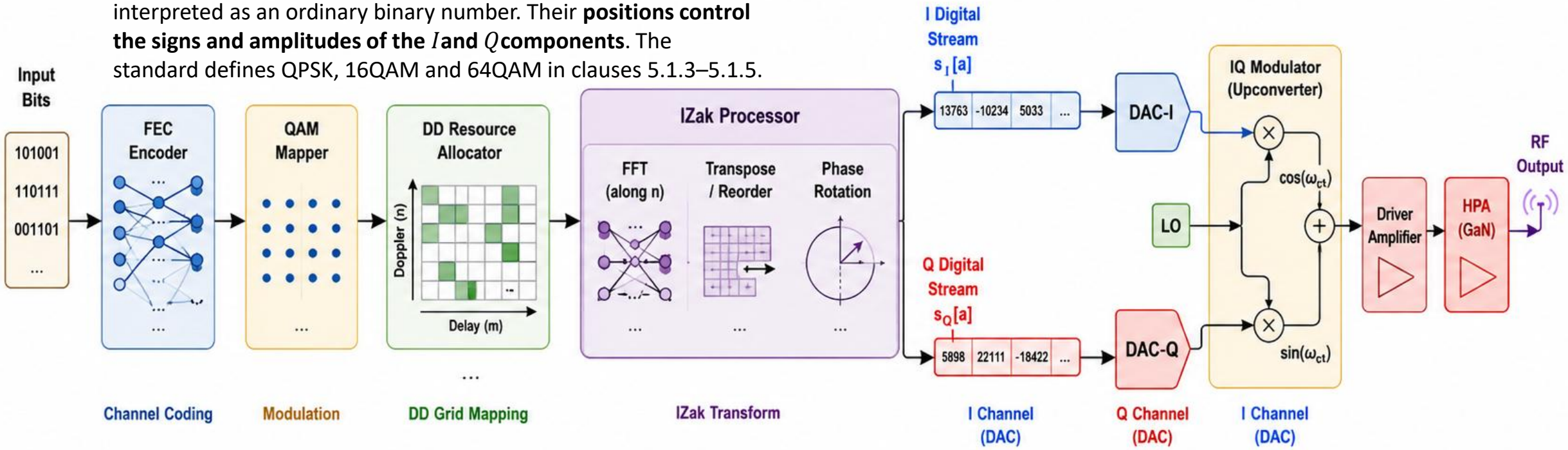
Therefore the number of hardware samples in one Zak period is

$$\frac{T}{T_{\text{samp}}} = \frac{MT_s}{T_s/D} = DM = \text{P}$$

So: $DM =$ number of DAC samples corresponding to one Zak period T.

NATIVE OTFS TRANSMITTER – SIMPLE ARCHITECTURE (IZAK WITH I/Q OUTPUT)

Use the 3GPP TS 38.211 convention, the input bits are not interpreted as an ordinary binary number. Their **positions control the signs and amplitudes of the $I$and Qcomponents**. The standard defines QPSK, 16QAM and 64QAM in clauses 5.1.3–5.1.5.



1. FEC Encoder
- LDPC / Polar / BCH
- Adds redundancy
- Output: coded bits

2. QAM Mapper
- Maps bits to complex symbols (e.g., 64QAM)
- Output: X[m,n]

3. DD Resource Allocator
- Maps symbols to Delay–Doppler bins
- Forms DD grid X[m,n]

4. IZak Processor
- FFT along Doppler (n)
- Transpose / Reorder
- Phase rotation
- Output: time-domain complex samples s[a]

5. I Channel (DAC)
- I digital samples $s_I[a]$
- 16-bit words
- DAC converts to analog I(t)

6. Q Channel (DAC)
- Q digital samples $s_Q[a]$
- 16-bit words
- DAC converts to analog Q(t)

7. IQ Modulator
- Upconverts I(t) and Q(t) to RF
- $s_{RF}(t) = I(t)\cos(\omega_{ct}) - Q(t)\sin(\omega_{ct})$

8. Driver + HPA
- Drives the high power amplifier
- Produces final RF output

DATA FORMAT (EXAMPLE)

Bits (0 / 1)	QAM Symbol (Example 64QAM)	DD Grid Value X[m,n]	IZak Output Sample s[a] = I + jQ	I Sample (16-bit) Fixed Point	Q Sample (16-bit) Fixed Point	Analog RF (After Upconversion)
101001 ...	3 - j5	3 - j5	0.42 + j0.18	13763 (0x35C3)	5898 (0x170A)	

Nature of g(t)

Nature of g(t)– Gaussian Sync function

for reducing PAPR
There are other functions and applications also

$$g(t) = \frac{1}{\sqrt{C_{\text{norm}}}} \cdot e^{-\pi\alpha\left(\frac{t}{T}\right)^2} \cdot \text{sinc}\left(\frac{t}{\tau_s}\right)$$

$$C_{\text{norm}} = \int_{-\infty}^{\infty} e^{-2\pi\alpha\left(\frac{t}{T}\right)^2} \cdot \text{sinc}^2\left(\frac{t}{\tau_s}\right) dt$$

$$\text{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$$

as $\int_{-\infty}^{\infty} |g(t)|^2 \, dt = 1$

$$\int_{-\infty}^{\infty} |g(t)|^2 \, dt = 1$$

IZak computation load

Using only the fixed transmitter-side equations:

$$v_m[q] = \frac{1}{N}\sum_{n=0}^{N-1} X\,[m, n]e^{j2\pi nq/N}$$

$$s[a] = \sum_{m=0}^{M-1}\sum_{q=0}^{N-1} v_m\,[q]\,g_{\mathrm{tx}}[a-(mD+Pq)]$$

With

$$L_g = \text{number of retained sampled taps of } g_{\mathrm{tx}}[\cdot]$$

So computation has two main parts.

1. Inner IZak / IFFT computation

For each fixed m:

$$v_m[q] = \frac{1}{N}\sum_{n=0}^{N-1} X\,[m, n]e^{j2\pi nq/N}$$

This is an N-point IFFT over n:

$$\boxed{v_m[:] = \mathrm{IFFT}_N\{X[m, :]\}}$$

There are M delay rows.

FFT-accelerated computation:

$$\boxed{C_{\mathrm{IZak,FFT}} = M\frac{N}{2}\log_2 N}$$

FFT butterflies per frame.

If computed directly without IFFT:

$$\boxed{C_{\mathrm{direct}} = MN^2}$$

complex MACs, which is much larger for large N.

IZAK computation load

$v_m[:] = \mathrm{IFFT}\{X[m,:]\}$, what the notation generally mean

The notation

$$\boldsymbol{v_m[:] = \mathbf{IFFT}\{X[m,:]\}}$$

is merely a shorthand for writing all Nequations simultaneously.

In expanded form:

$$v_m[0] = \frac{1}{N}\sum_n X\,[m,n]e^{j2\pi n\cdot 0/N}$$

$$v_m[1] = \frac{1}{N}\sum_n X\,[m,n]e^{j2\pi n\cdot 1/N}$$

$$v_m[2] = \frac{1}{N}\sum_n X\,[m,n]e^{j2\pi n\cdot 2/N}$$

$$\vdots$$

$$v_m[N-1] = \frac{1}{N}\sum_n X\,[m,n]e^{j2\pi n(N-1)/N}$$

$v_m[:]$ means the **entire vector**.

$$v_m[:] = \begin{bmatrix} v_m[0] \\ v_m[1] \\ v_m[2] \\ \vdots \\ v_m[N-1] \end{bmatrix}$$

Computation load of IZak

2. Pulse synthesis using $g_{\mathrm{tx}}(t)$

Now each coefficient $v_m[q]$multiplies a shifted transmit pulse:

$$g_{\mathrm{tx}}(t - mT_s - qT)$$

In sampled hardware:

$$s[a] = \sum_{m=0}^{M-1} \sum_{q=0}^{N-1} v_m[q]\, g_{\mathrm{tx}}[a - mD - qP]$$

where:

$$D = \frac{T_s}{T_{\mathrm{samp}}}$$

$$P = \frac{T}{T_{\mathrm{samp}}} = \mathrm{MD}$$

If the effective pulse length is:

$$L_g$$

samples, then each (m, q)coefficient needs approximately $L_g$complex MACs for pulse synthesis.

Each $v_m[q]$multiplies $L_g$retained samples of $g_{\mathrm{tx}}[\cdot]$. There are

$$MN$$

values of $v_m[q]$.

So pulse synthesis costs: $C_{\mathrm{pulse}} = MNL_g$ complex MACs per frame.

If $g_{\mathrm{tx}}[\cdot]$ is purely real, this is complex-times-real MACs. If the pulse is complex, it is full complex MAC

Total transmitter-side load

Direct IZak - $C_{\mathrm{tx,direct}} = MN^2 + MNL_g$

FFT-accelerated IZak $\boldsymbol{C_{\mathrm{tx,FFT}} = M\frac{N}{2}\log_2 N + MNL_g}$

$L_g$is relevant only in the second term, the pulse-synthesis term.

Computation Load of IZak

Numerical value for $M = N = 128$

$$MN = 128 \times 128 = 16384.$$

Direct IZak:

$$C_{\text{IZak,direct}} = 128 \times 128^2 = \boxed{2{,}097{,}152}$$

complex MACs per frame.

FFT IZak:

$$C_{\text{IZak,FFT}} = 128 \times 64 \times 7 = \boxed{57{,}344}$$

butterflies per frame.

Pulse synthesis:

$$\boxed{C_{\text{pulse}} = 16384 L_g}$$

complex MACs per frame.

For the conservative case

$$L_g = 31,$$

$$\boxed{C_{\text{tx,FFT}} = 57{,}344 + 507{,}904 = 565{,}248}$$

per frame.

Per-second load for $B = 500\,\text{MHz}$

Frame rate:

$$\boxed{f_{\text{frame}} = \frac{B}{MN}}$$

For

$$B = 500\,\text{MHz}, M = N = 128,$$

$$f_{\text{frame}} = \frac{500 \times 10^6}{16384} = \boxed{30{,}517.6\ \text{frames/s}}.$$

FFT IZak rate

$$R_{\text{IZak,FFT}} = M \frac{N}{2} \log_2 N \frac{B}{MN} = \boxed{\frac{B}{2} \log_2 N}$$

$$= \frac{500 \times 10^6}{2} \times 7 = \boxed{1.75\ \text{Gbutterflies/s}}.$$

Pulse-synthesis rate

$$R_{\text{pulse}} = MNL_g \frac{B}{MN} = \boxed{BL_g}$$

For $L_g = 31$:

$$R_{\text{pulse}} = 500 \times 10^6 \times 31 = \boxed{15.5\ \text{Gcomplex MAC/s}}.$$

Computation Load of IZak

Total per-second transmitter load

Direct IZak

$$\boxed{R_{\text{tx,direct}} = B(N + L_g)}$$

For $L_g = 31$:

$$R_{\text{tx,direct}} = 500 \times 10^6(128 + 31) = \boxed{79.5 \text{ Gcomplex MAC/s}}.$$

FFT-accelerated IZak

$$\boxed{R_{\text{tx,FFT}} = BL_g + \frac{B}{2}\log_2 N}$$

For $L_g = 31$:

$$\boxed{R_{\text{tx,FFT}} = 15.5 \text{ Gcomplex MAC/s} + 1.75 \text{ Gbutterflies/s}.}$$

Important role of D

The transform count does not explicitly contain D:

$$\boxed{C_{\text{IZak,FFT}} = M\frac{N}{2}\log_2 N.}$$

But D affects the pulse-synthesis load indirectly because L_g is measured in DAC samples.

If D increases, then

$$T_{\text{samp}} = \frac{T_s}{D}$$

becomes smaller, and the same physical pulse occupies more samples. So L_g usually increases.

Therefore:

$$\boxed{\begin{array}{c} D \text{ affects computation through } L_g, \\ \text{not through the IZak row transform.} \end{array}}$$

IZak and Zak in Tx and Rx

Use this receive definition:

$$u_m[q] = \int r\,(t)\, g_{\mathrm{rx}}(t - mT_s - qT)\, dt$$

With $g_{\mathrm{rx}} = g_{\mathrm{tx}}^*$

$$Y[m, n] = \sum_{q=0}^{N-1} u_m\,[q] e^{-j2\pi nq/N}$$

$$\boxed{r(t) \rightarrow g_{\mathrm{rx}}^* \rightarrow u_m[q] \rightarrow \text{FFT over q} \rightarrow Y[m, n]}$$

DD Memory X[m,n]

↓

N-point IFFT

↓

v_m[q]

↓

Pulse synthesis g_tx

↓

Channel

↓

Matched filter g_rx

↓

u_m[q]

↓

N-point FFT

↓

Y[m,n]

Transmit side:

$$v_m[q] = \frac{1}{N} \sum_{n=0}^{N-1} X\,[m, n] e^{+j2\pi nq/N}$$

$$s(t) = \sum_m \sum_q v_m\,[q] g_{\mathrm{tx}}(t - mT_s - qT)$$

$$\boxed{X[m, n] \rightarrow \text{IFFT over } n \rightarrow v_m[q] \rightarrow g_{\mathrm{tx}} \rightarrow s(t)}$$

DIFFERENCE BETWEEN TWO USER EQUIPMENT (UE)

CURRENT UE (OFDM BASED)

UE-OFDM

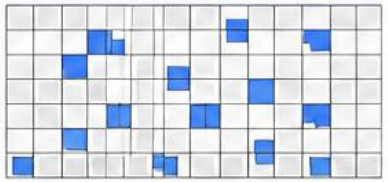
Focus: Throughput & Coverage
Doppler is an impairment to be compensated

Bits → FEC → QAM → OFDM (IFFT + CP) → DAC → RF Front End

RF Front End → ADC → OFDM (FFT) → Equalizer → QAM Demap → FEC Decode → Bits

SIGNAL PROCESSING VIEW

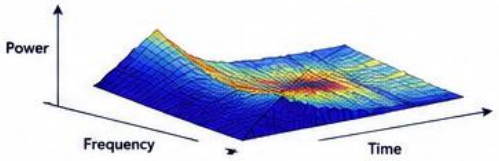
Time–Frequency Grid (Resource Elements)



Characteristics
- Subcarriers orthogonal in frequency
- Time slots orthogonal in time
- Doppler causes ICI
- Needs CP and channel estimation
- Mobility compensation required

MOBILITY & CHANNEL VIEW

Channel Effect in High Mobility



Impact
- High Doppler → severe ICI
- Performance drops
- Needs more pilots
- Handover harder

CAPABILITIES OF UE-OFDM

High Throughput	Mature Ecosystem	Limited Mobility Robustness	No Native Sensing	Lower Processing Demand
Good spectral efficiency in low mobility	Proven, widely deployed, cost optimized	Doppler is an impairment	Not designed for CNS (Communication, Navigation, Sensing)	FFT based, less complex

SUMMARY – UE-OFDM
- Designed for spectral efficiency in relatively low mobility.
- Doppler is a problem to be mitigated.

VS

FUTURE UE (OTFS/DD BASED)

UE-OTFS

Focus: Mobility, DD Awareness & CNS
Doppler is a native dimension (resource) to be used

BFN IC

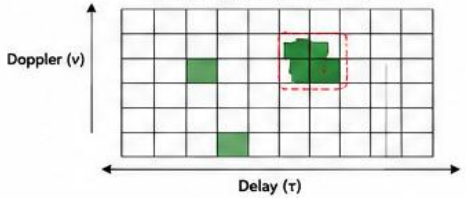
Bits → FEC → QAM → DD Mapper (Delay–Doppler) → OTFS IZak with pulse shaping → DAC → RF Front End

RF Front End → ADC → OTFS Zak with matched filter → DD Equalizer → QAM Demap → FEC Decode → Bits

OTFS Modem IC

SIGNAL PROCESSING VIEW

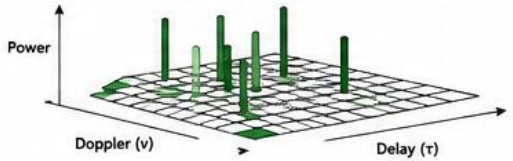
Delay–Doppler Grid (DD Bins)



Characteristics
- Resources in Delay–Doppler domain
- Doppler handled as a dimension
- Excellent high mobility performance
- Less sensitive to Doppler spread
- Sparse channel in DD domain

MOBILITY & CHANNEL VIEW

Channel Representation in DD Domain



Impact
- Channel is sparse in DD
- Robust to high Doppler
- Fewer pilots needed
- Smoother handover

CAPABILITIES OF UE-OTFS/DD

High Mobility Robustness	DD Self-Localization	CNS Support	DD-aware Beam Tracking	Higher Processing Capability
Doppler as a native dimension	UE can estimate its own delay/Doppler (position/velocity)	Enables Communication, Navigation and Sensing	Better beam selection and handover	New modem, more powerful

SUMMARY – UE-OTFS/DD
- Designed for high mobility, DD domain operation and CNS.
- Doppler is a dimension to be used, not an impairment.

USER MOBILE Handset - OTFS Transmitter ICs Example

Bits
↓
FEC Encoder
↓
QAM Mapper
↓
DD Resource Allocator
↓
DD Grid X[m,n]
↓
IZak Processor
↓
Pulse Shaping g(t)
↓
Digital Predistortion (optional)
↓
DAC
↓
Reconstruction Filter
↓
IQ Upconverter
↓
Driver Amplifier
↓
HPA

1. FEC Engine
Input:
`Bits`
Output:
`Coded bits`

Hardware:
•LDPC ASIC
•Polar ASIC
•BCH/RS (if desired)
Can be reused from OFDM systems.

2. QAM Mapper
Input:
`6 bits`
Output:

$$c_k$$

Example:

$$101001 \rightarrow 5 + 3j$$

Hardware:
•small lookup table
•trivial logic
Can be reused.

3. DD Resource Allocator
This is OTFS-specific.
Input:
`User symbols`
Output:

$$X[m, n]$$

Hardware:
`Scheduler`
`Memory Controller`
`DD Bin Allocation Table`
This is one of the new OTFS ASIC functions.

4. DD Memory
Stores:

$$X[m, n]$$

Example:
`256 × 256 complex symbols`

Hardware:
`SRAM`
`DDR`
`HBM`

5. IZak Processor
This is the heart of native OTFS.
Input:

$$X[m, n]$$

Output:

$$s[a]$$

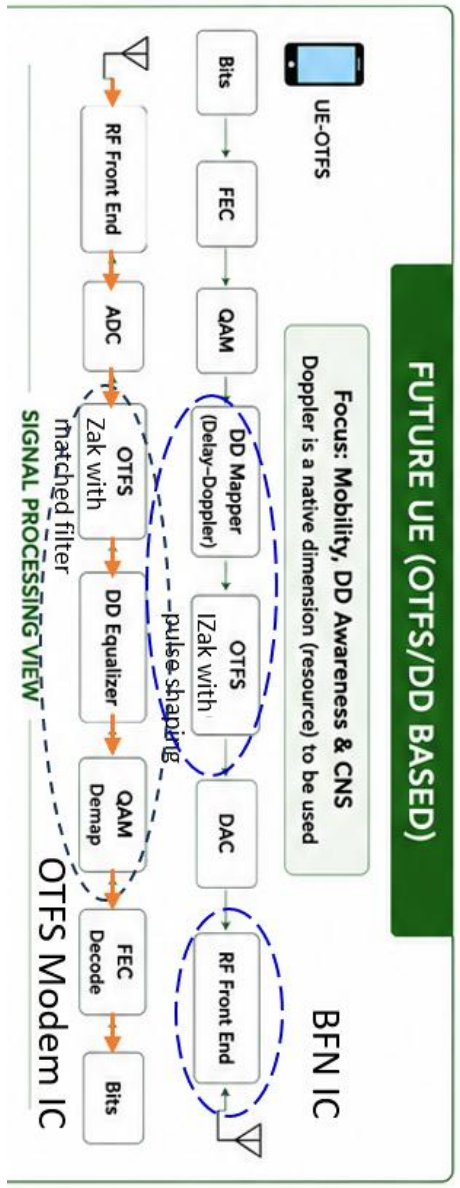
Hardware internally may use:
`FFT Engines`
`Memory Transpose`
`Phase Rotators`
even though architecturally it is IZak.
This is the second major OTFS-specific ASIC.

6. Pulse Shaping Engine
Implements:

$$g(t)$$

Examples:
•Gaussian
•Orthogonalized Gaussian
•IOTA

Hardware:
`FIR Filter`
`Polyphase Filter Bank`



Bits
↓
FEC Encoder
↓
QAM Mapper
↓
DD Resource Allocator
↓
DD Grid X[m,n]
↓
IZak Processor
↓
Pulse Shaping g(t)
↓
Digital Predistortion (optional)
↓
DAC
↓
Reconstruction Filter
↓
IQ Upconverter
↓
Driver Amplifier
↓
HPA

OTFS Transmitter

7. DPD (optional)
For GaN HPA.

Hardware:
`LUT`
`Adaptive DSP`
Can be reused from OFDM.

8. DAC
Input:

$$s[a]$$

Output:
analog waveform.

Hardware:
`High-speed DAC`
Can be reused.

9. Reconstruction Filter
Removes DAC images.
Hardware:
`LPF / BPF`

10. IQ Upconverter
Hardware:
`LO`
`IQ Mixer`
Can be reused.

11. Driver + HPA
Hardware:
`Driver Amp`
`GaN HPA`
Can be reused.

Layer	Complete loopback included
0	Exact numerical IZak → Zak
1	Finite sampled g_{tx} and g_{rx}
2	DAC/ADC sampling, quantization and linear reconstruction filters
3	Output mask filter with PA bypassed
4	PA followed by the output mask filter
5	One integer delay–Doppler path
6	One fractional $\Delta m, \Delta n$ path
7	Dense physical multipath
8	Pilot-based SISO channel estimation and equalization
9	Ideal MIMO spatial channel
10	Fractional dense-multipath MIMO channel
11	MIMO pilot estimation and equalization
12	Complete RF and propagation loopback

Start With Level 0 Loopback

$$X[m,n] \xrightarrow{\text{IZak}} s[a] \xrightarrow{\text{Zak}} Y[n,n] = X[m,n].$$

Then the complete chain

At the Transmitter,

$$\text{Speech/Data} \rightarrow \text{PCM bits} \rightarrow 4\text{–bit 16QAM labels} \rightarrow \text{DD memory} \rightarrow \text{3GPP mapper} \rightarrow X_{\mathrm{DD}}(\tau,\nu) \rightarrow \text{IZak} \rightarrow I[n], Q[n] \rightarrow \text{I/Q DAC} \rightarrow \text{RF transmission.}$$

At the receiver,

$$\text{received RF} \rightarrow I/Q\ \text{ADC} \rightarrow y[n] \rightarrow \text{Zak} \rightarrow \text{DD equalization} \rightarrow \text{16QAM demapping} \rightarrow \text{PCM words} \rightarrow \text{reconstructed Data/Speech.}$$

Next level 1 onwards

Throughput
Multiple Access
Routing

dd bins – limit for M of M,N

B is the bandwidth

$$\text{Delay sampling interval Ts or } \Delta\tau = \frac{1}{B}$$

$\Delta\tau$ or Ts is already fixed by bandwidth B

Then:

$$T = M\Delta\tau = \frac{M}{B}.$$

Now Doppler phase is sampled every T: $e^{j2\pi\nu qT}$.

Sampling theory then imposes:

$$|\ \nu\ | < \frac{1}{2T}$$

to avoid Doppler aliasing.

$$T \leq \frac{1}{2\nu_{max}}$$

Substituting:

$$T = \frac{M}{B},$$

gives:

$$\boxed{M < \frac{B}{2\nu_{\max}}}$$

This is OTFS/Zak lattice constraint.

So for fixed bandwidth:

- delay resolution fixed,
- increasing M only enlarges delay window,
- but eventually Doppler aliases.

Therefore: maximum unamiguous Doppler fundamentally limits M

The margins should consider following parameters also:
positive and negative Doppler support; acceleration during the frame; ephemeris error; oscillator error; phase-noise broadening; leakage skirts.

dd bins – limit for M of M,N – bandpass doppler sampling

Example with NTN Doppler bands
Suppose choose:

$$\frac{1}{T} = 8 \text{ kHz}$$

instead of 30 kHz.
Then Doppler aliases modulo: 8 kHz.

Now:
Low band $0–1.5$ kHz
stays near DC.

High band $12–15$ kHz

aliases to: $4–7$ kHz
because:

$$12 - 8 = 4$$
$$15 - 8 = 7.$$

Now bands remain non-overlapping: $0–1.5$
And $4–7$ kHz.: Perfectly separable.

What does this do to M?
Now:

$$T = \frac{1}{8 \text{ kHz}} = 125\mu s$$

instead of:

$$33\mu s.$$

Thus:

$$M = BT = 500 \times 10^6 \times 125 \times 10^{-6}$$
$$\boxed{M = 62500}$$

Now:
•larger delay window,
•still no Doppler ambiguity,
provided Doppler bands remain separable after aliasing.

•Instead of

$$T = \frac{1}{30 \text{ kHz}} = 33\mu s$$

Which limits to M = 16666.

dd bins – Limit for N of M,N

$Zak\ or\ OTFS\ Period\ T \leq \frac{1}{2\nu_{max}}$ to avoid Doppler aliasing.

Then choose: $T_f = NT$
to achieve desired Doppler resolution.

Finally verify:
$T_f \lesssim T_{coh}, T_{coh} \rightarrow$ Channel validity
Channel is approximately unchanged

The fundamental limit on N is:

$$T_f = NT \leq T_{\text{coh}}$$

where:

T_{coh}
$=$ time over which the channel remains predictably coherent

Therefore:

$$N_{\max} = \frac{T_{\text{coh}}}{T} = \frac{BT_{\text{coh}}}{M}$$

What determines T_{coh}?

T_{coh}
is limited by:
Doppler variation, acceleration, oscillator phase noise, timing drift, and channel nonstationarity

So:

N is limited by how long the Doppler phase law remains coherent.

Separately, N has a lower requirement from desired Doppler resolution:

$$\Delta\nu = \frac{1}{NT}$$

so:

$$\boxed{N_{\min} = \frac{1}{T\Delta\nu_{\text{desired}}}}$$

Thus:

$$\boxed{\frac{1}{T\Delta\nu_{\text{desired}}} \leq N \leq \frac{T_{\text{coh}}}{T}}$$

M and N has direct implication on accommodating number of users, i.e., Multiple Access

Throughput

In OTFS DD-bin language, throughput comes directly from:

number of payload DD bins transmitted per second

Let:

$$M = \text{number of delay bins}$$
$$N = \text{number of Doppler bins}$$

Then total DD bins per OTFS frame:

$$\boxed{MN}$$

Now suppose:

•only a fraction used for payload:

$$\eta_{\text{DD}} = \frac{\text{payload DD bins}}{MN}$$

because some bins reserved for:

•pilots,

•guards,

•sensing,

•control.

Then payload DD bins per frame:

$$N_{\text{payload}} = \eta_{\text{DD}} MN$$

1. Bits carried by one DD bin

For modulation order:

$$Q$$

bits per symbol:

$$b = \log_2(Q)$$

For 64QAM: $b = 6$

So one payload DD bin carries:

$$6 \text{ bits}$$

(before coding).

2. OTFS frame duration

One OTFS period:

$$T = \frac{M}{B}$$

Total frame duration:

$$\boxed{T_{\text{frame}} = NT = \frac{MN}{B}}$$

Throughput

3. Payload bits per frame

$$\boxed{N_{\text{bits/frame}} = \eta_{\text{DD}} MN\, b}$$

4. Throughput

$$R = \frac{N_{\text{bits/frame}}}{T_{\text{frame}}}$$

Substitute:

$$R = \frac{\eta_{\text{DD}} MN\, b}{MN/B}$$

The MN cancels:

$$R = \eta_{\text{DD}}\, B\, b$$

This is the fundamental OTFS throughput equation.

5. Including coding

If FEC code rate:

$$r_c$$

then:

$$R_{\text{payload}} = \eta_{\text{DD}}\, r_c\, B\, b$$

6. Important insight

Notice: M and N cancel out

Therefore:

- M, N determine DD geometry/resources,
- but not ideal spectral efficiency.

They affect throughput indirectly through:

- pilot overhead,
- guards,
- sparsity,
- scheduling,
- latency,
- user multiplexing efficiency.

But Shannon-style throughput (ideal packed MCS payload rate) depends only on:

- bandwidth,
- modulation order,
- coding,
- payload occupancy fraction.

Throughput and dd resource allocation – Multiple Access

7. Satellite (NTN) example

Per beam:

$$B = 500\ \text{MHz}$$

64QAM:

$$b = 6$$

Suppose:

•payload occupancy:

$$\eta_{\text{DD}} = 0.8$$

•code rate:

$$r_c = 0.8$$

Then:

$$R_{\text{beam}} = 0.8 \times 0.8 \times 500 \times 10^6 \times 6$$
$$R_{\text{beam}} \approx 1.92\ \text{Gbps}$$

For 48 beams:

$$R_{\text{total}} = 48 \times 1.92$$
$$R_{\text{total}} \approx 92\ \text{Gbps}$$

A useful formula for user is:

$$R_u = \frac{X\ \log_2(S_{\text{QAM}})\ R_c}{T_f}$$

Ideal total spectral throughput:

$$R \approx B \times \text{bits/s/Hz}$$

is fundamentally determined by:

•bandwidth,

•modulation order,

•coding,

•SNR.

8. OTFS-native interpretation

Each DD bin is:

•one delay-localized Doppler mode.

Throughput is:

how many payload DD modes are filled per second

This makes the basis of resource management in this form of multiple access

where:

•$X\ is\ X_{\text{DD,alloc}}$= average DD bins assigned to user,

•S_{QAM}= modulation order,

•R_c= coding rate,

•T_f= OTFS frame duration.

Throughput and dd resource allocation – Multiple Access

Control bits in user dd location, can carry addresses of other dd bins allocated to user for payload data also with other control information, to help switching and payload data aggregation.

$$\boxed{\text{user DD location} = \text{control anchor}}$$

Control anchor carries:

addresses/pointers to payload DD regions

plus:

- sender ID, receiver ID, modulation/coding,
- sequence number, aggregation rule,
- switching/routing metadata, validity time, error check.

The route information has to be available before the associated payload block is committed to an outgoing beam or ISL.

Then payload bins elsewhere do not need full headers. They only need either:

- no header, if listed in anchor, or
- short pointer/tag if fragmented.

So the structure becomes:

$$\text{DD anchor} \rightarrow \{\mathcal{R}_{u,1}, \mathcal{R}_{u,2}, \ldots \mathcal{R}_{u,K}\} \rightarrow \text{payload aggregation}$$

where:

$$\mathcal{R}_{u,k} = \text{DD payload block assigned to user } u.$$

This helps switching because the satellite/base station can read only the control anchor and know:

which DD payload regions belong to which user/flow

without demodulating all payload content.

So the model is:

sparse DD control anchors + dense DD payload packing

That gives:

- low control overhead,
- high payload occupancy,
- clean aggregation,
- payload-transparent switching,
- scalable DD routing.

Throughput and dd resource allocation – Multiple Access

beacon sent by base station → DD estimate by all active users → DD−state report sent ***by active users to base station*** → Base station creates DD map − DD allocation ***by base station to users*** → user fills allocated dd bins with payload

Minimal report: user ID, delay bin, Doppler bin, confidence/SNR, timestamp

Or: user ID, $m_u, n_u, \Delta m, \Delta n,$ drift, SNR, beam ID

How to send it:

1.Initial access: user sends a short DD-state report in a reserved uplink control anchor region.

2.Tracking: user periodically sends small updates only when DD position changes beyond threshold.

3.Prediction: reference station extrapolates between updates using drift:

$$m_u(t + \Delta t) \approx m_u(t) + \dot{m}_u \Delta t$$
$$n_u(t + \Delta t) \approx n_u(t) + \dot{n}_u \Delta t$$

4.Allocation: station writes payload-bin addresses into that user's DD control anchor.

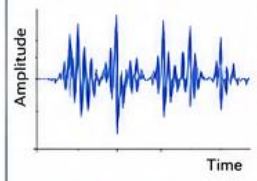
END–TO–END OTFS COMMUNICATION LINK

From Sender_1 to Receiver_1

SENDER_1

1. Information (Time Domain)

Original signal $x(t)$

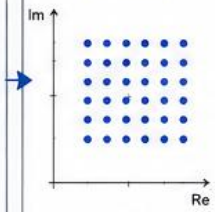


Example: voice



2. Modulation (Mapping)

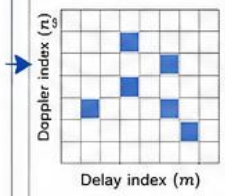
Symbols c_k (64QAM)



Each symbol carries 6 bits

3. Map to DD Domain

DD symbols $X[m,n]$

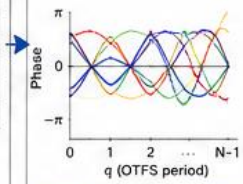


Allocated DD resources

4. Doppler Phase Synthesis

$v_m[q]$

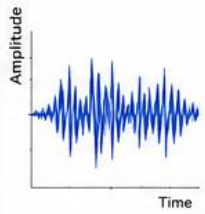
Phase evolution across OTFS periods (q)



Each Doppler bin creates a specific phase trajectory

5. Time Domain Waveform $s(t)$

OTFS transmit waveform



Sum of all time-shifted pulses $g_{tx}(t)$

6. RF Transmission



Upconversion, Amplification, Transmission



7. Propagation Channel (Delay–Doppler Nature)

Multipath + Doppler

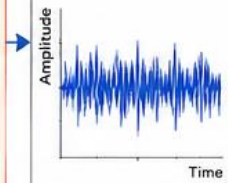


$$r(t) = \sum_p h_p s(t-\tau_p) e^{j2\pi v_p t} + w(t)$$

Delays (τ_p) and Dopplers (ν_p) added by the channel

8. Received Signal $r(t)$

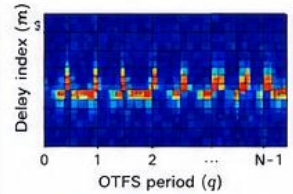
Time domain received signal



RECEIVER_1

9. DD Projection (Delay Extraction)

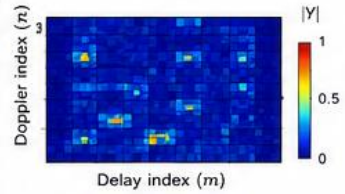
$u_m[q]$



Correlate with receive pulses $g_{rx}(t-m\Delta\tau-qT)$

10. Doppler Extraction (Doppler FFT)

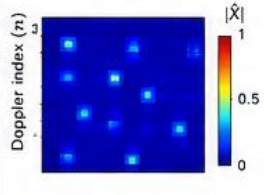
$Y[m,n]$



DD domain received symbols

11. DD Equalization

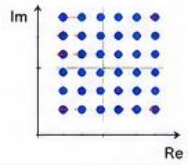
$\hat{H}[m,n]$ estimated → Equalize



Compensate channel effect in DD domain

12. Demapping & Decoding

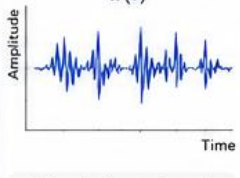
Recovered symbols $\hat{c}_k$ and bits $\hat{b}_k$



Demap 64QAM → bits → decode

13. Reconstruct (Time Domain)

Recovered signal $\hat{x}(t)$

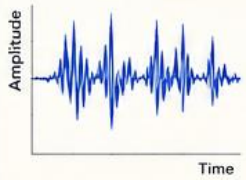


Output: time domain signal at Receiver_1

DOMAINS AND THEIR TYPICAL OUTPUT APPEARANCE

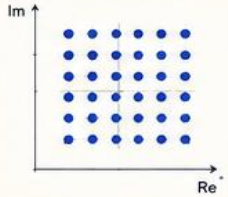
Time Domain (original / reconstructed)

$x(t)$ or $\hat{x}(t)$



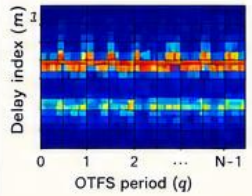
Constellation Domain (modulation symbols)

64QAM example



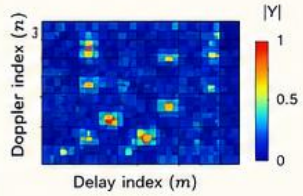
Delay Domain (after projection)

$u_m[q]$ (vs q for each m)



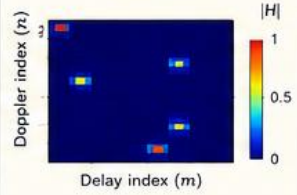
Doppler Domain (after Doppler FFT)

$Y[m,n]$



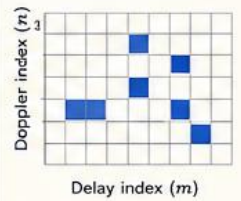
Channel DD Response (sparse in DD)

$H[m,n]$



DD Resource Grid (OTFS frame)

$X[m,n]$ allocation



KEY INSIGHT

Information is embedded into Delay–Doppler (DD) resources.

↓

Channel adds delay and Doppler shifts.

↓

Receiver observes in DD and equalizes.

↓

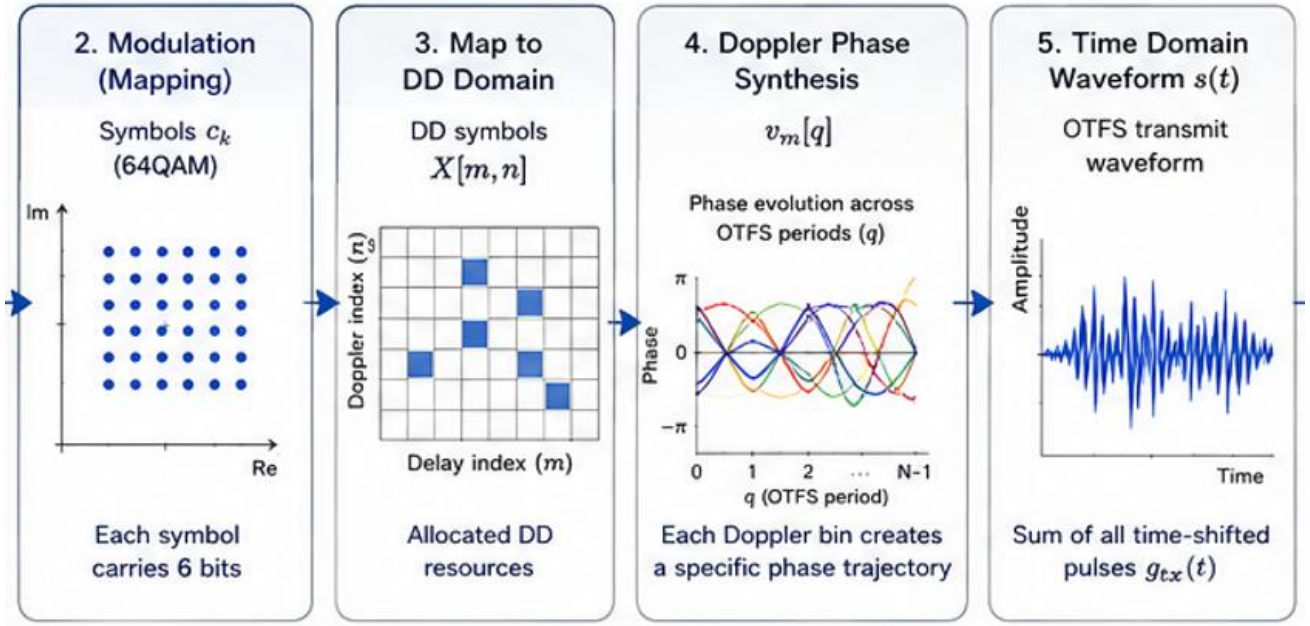
Original time-domain signal is recovered.

OTFS communicates by encoding data into the natural language of the wireless channel: Delay and Doppler.

How user fills allocated dd bins

beacon sent by base station → DD estimate by all active users → DD−state report sent ***by active users to base station*** → Base station creates DD map − DD allocation ***by base station to users*** → user fills allocated dd bins with payload

For Transmission, How following steps are implemented in silicon/software:



QAM symbols → DD grid SRAM → IZAK Engine/ → s(t) / DAC

How QAM symbols are loaded in DD bins

Suppose the transmitter loading symbols for QAM

$$c_0, c_1, c_2, \ldots, c_{63}$$

In First allocation

The transmitter loads:

$$X[m_1, n_1] = c_0$$
$$X[m_2, n_2] = c_1$$
$$\cdots$$
$$X[m_{10}, n_{10}] = c_9$$

That is one OTFS frame.

Subsequent allocation may have different number of dd bins

A useful formula for user is:

$$R_u = \frac{X \log_2(S_{\mathrm{QAM}}) R_c}{T_f}$$

First allocation

10 DD bins allocated.

Each bin carries one 64QAM symbol.

Therefore:

$$10 \text{ symbols/frame}$$

Symbol rate:

$$R_s = \frac{10}{T_f} = 10{,}000 \text{ symbols/s}$$

Next allocation

16 DD bins allocated.

Now:

$$16 \text{ symbols/frame}$$

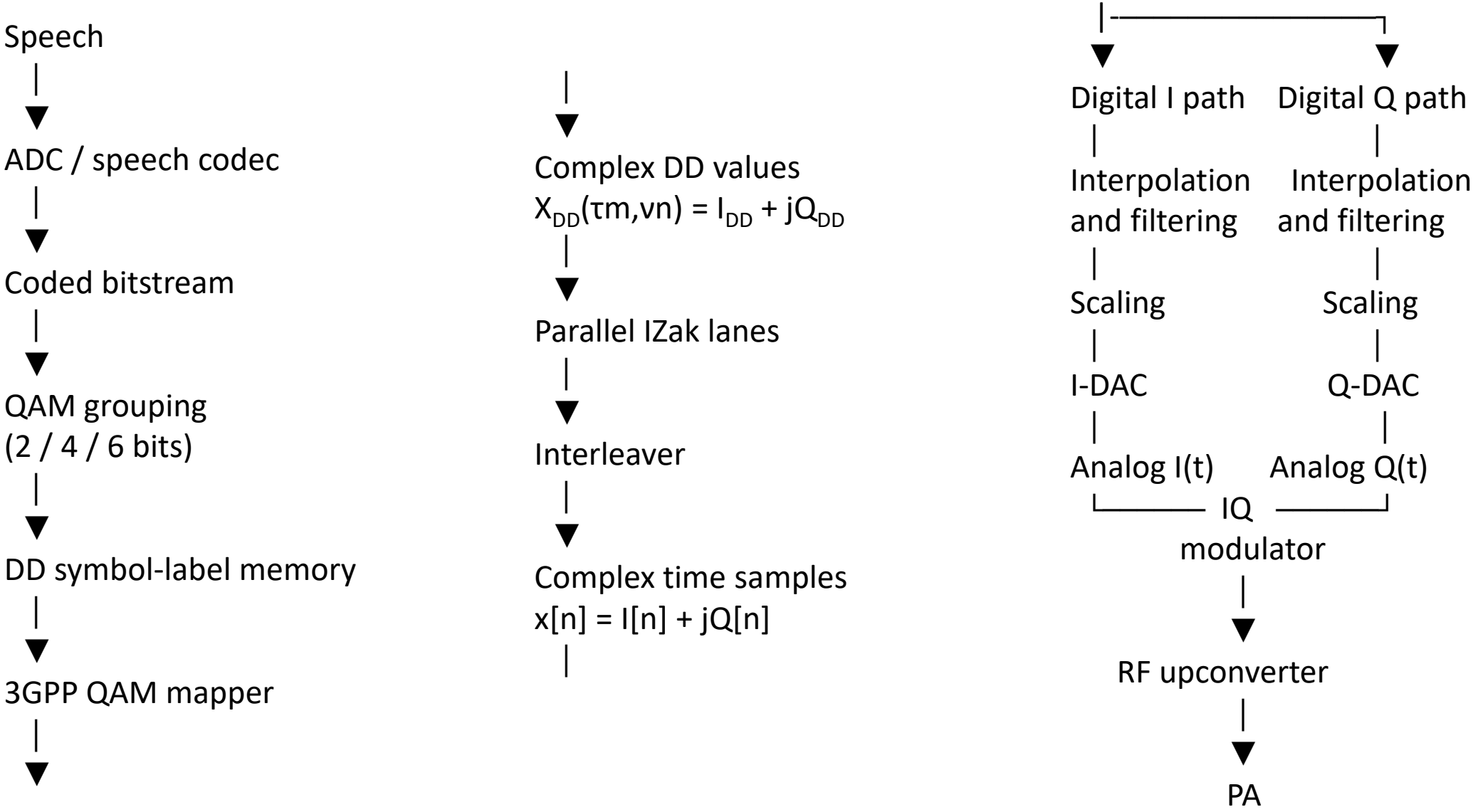
Symbol rate becomes:

$$R_s = \frac{16}{T_f} = 16{,}000 \text{ symbols/s}$$

where:

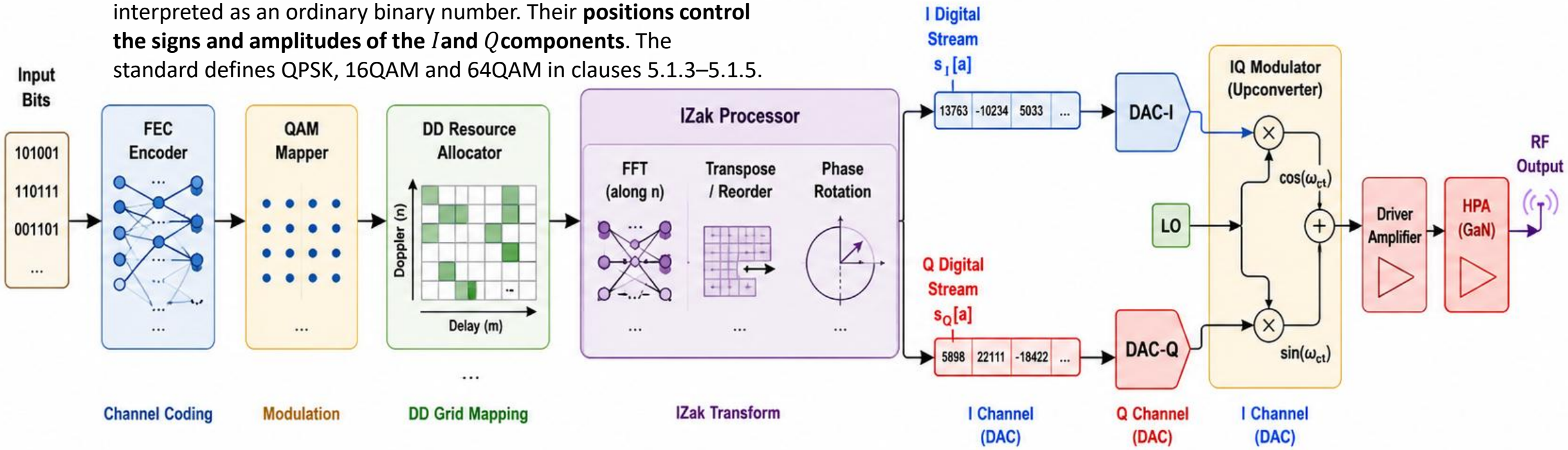
- $X \; is \; X_{\mathrm{DD,alloc}}$= average DD bins assigned to user,
- S_{QAM}= modulation order,
- R_c= coding rate,
- T_f= OTFS frame duration.

QAM symbols to Modulation



NATIVE OTFS TRANSMITTER – SIMPLE ARCHITECTURE (IZAK WITH I/Q OUTPUT)

Use the 3GPP TS 38.211 convention, the input bits are not interpreted as an ordinary binary number. Their **positions control the signs and amplitudes of the $I$and Qcomponents**. The standard defines QPSK, 16QAM and 64QAM in clauses 5.1.3–5.1.5.



1. FEC Encoder
- LDPC / Polar / BCH
- Adds redundancy
- Output: coded bits

2. QAM Mapper
- Maps bits to complex symbols (e.g., 64QAM)
- Output: X[m,n]

3. DD Resource Allocator
- Maps symbols to Delay–Doppler bins
- Forms DD grid X[m,n]

4. IZak Processor
- FFT along Doppler (n)
- Transpose / Reorder
- Phase rotation
- Output: time-domain complex samples s[a]

5. I Channel (DAC)
- I digital samples $s_I[a]$
- 16-bit words
- DAC converts to analog I(t)

6. Q Channel (DAC)
- Q digital samples $s_Q[a]$
- 16-bit words
- DAC converts to analog Q(t)

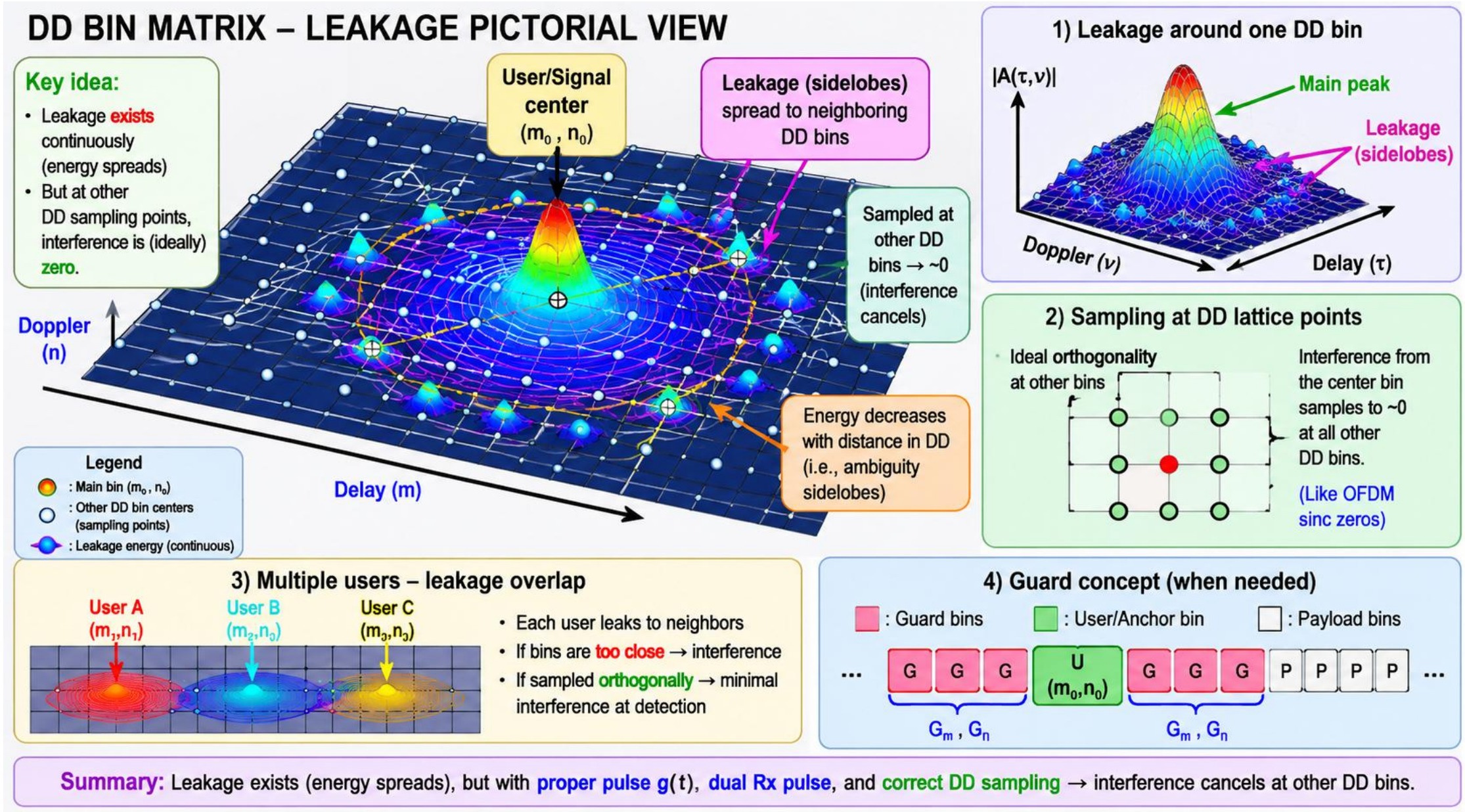
7. IQ Modulator
- Upconverts I(t) and Q(t) to RF
- $s_{RF}(t) = I(t)\cos(\omega_{ct}) - Q(t)\sin(\omega_{ct})$

8. Driver + HPA
- Drives the high power amplifier
- Produces final RF output

DATA FORMAT (EXAMPLE)

Bits (0 / 1)	QAM Symbol (Example 64QAM)	DD Grid Value X[m,n]	IZak Output Sample s[a] = I + jQ	I Sample (16-bit) Fixed Point	Q Sample (16-bit) Fixed Point	Analog RF (After Upconversion)
101001 ...	3 - j5	3 - j5	0.42 + j0.18	13763 (0x35C3)	5898 (0x170A)	

Minimization of leakage and near-zero sampled interference



Minimization of leakage and near-zero sampled interference

Choose dd lattice spacing

$$\Delta\tau = \frac{1}{B}$$

$$\Delta\nu = \frac{1}{NT}$$

and enforce the sampling relation:

$$\Delta\tau\Delta\nu \approx \frac{1}{MN}$$

The lattice must be matched to bandwidth and observation time.

The practical objective is: Sampled Power

$$\boxed{A_{g_{\mathrm{rx}},g_{\mathrm{tx}}}(k,\Delta\tau,l\Delta\nu) \approx \delta[k]\delta[l]}$$

while keeping:

$$A_{g_{\mathrm{rx}},g_{\mathrm{tx}}}(\tau,\nu)$$

small between samples.

That gives:

overlapping DD modes, but near−zero sampled interference

Use:

$$g_{\mathrm{tx}}(t)$$

and

$$g_{\mathrm{rx}}(t)$$

as a matched/dual pair.

The condition is:

$$A_{g_{\mathrm{rx}},g_{\mathrm{tx}}}(k,\Delta\tau,l\Delta\nu) = 0$$

for all nonzero lattice offsets:

$$(k,l) \neq (0,0)$$

and:

$$A_{g_{\mathrm{rx}},g_{\mathrm{tx}}}(0,0) = 1.$$

where:

$$A_{g_{\mathrm{rx}},g_{\mathrm{tx}}}(\tau,\nu) = \int g_{\mathrm{rx}}^{*}(t-\tau)g_{\mathrm{tx}}(t)e^{j2\pi\nu t}dt.$$

Overlapping Zak DD modes remain distinguishable when the complete transmit and receive DD basis satisfies the required sampled biorthogonality condition.

Minimization of leakage and near-zero sampled interference – Further check on Zak-synthesized carrier pair

The imposed ambiguity-function condition directly on $g_{\mathrm{tx}}$and g_{rx}at the DD spacings:

$$A_{g_{\mathrm{rx}},g_{\mathrm{tx}}}(k,\Delta\tau,l\Delta\nu) \approx \delta[k]\delta[l].$$

That is a useful pulse-localization indicator,

But the actual transmitted DD mode is

$$\phi_{m,n}(t) = \frac{1}{N}\sum_{q=0}^{N-1} e^{+j2\pi nq/N}\, g_{\mathrm{tx}}(t - mT_s - qT).$$

The receive mode is correspondingly

$$\psi_{m,n}(t) = \sum_{q=0}^{N-1} e^{-j2\pi nq/N}\, g_{\mathrm{rx}}(t - mT_s - qT),$$

up to the selected normalization convention.

The actual no-interference or biorthogonality requirement is therefore

$$\langle \psi_{m,n'}\, \phi_{m',n'} \rangle \approx \delta_{m,m'}\delta_{n,n'}.$$

The elementary pulse ambiguity function may be used to help design this condition, but the complete Zak-synthesized carrier pair must be checked.

Choice and Nature of g(t)

Candidate well-localized prototype pulses: orthogonalized Gaussian/IOTA-type and PHYDYAS-type pulses

A commonly used PHYDYAS/IOTA-style orthogonalized prototype pulse can be written as:

$$g(t) = \sum_{k=0}^{K-1} \tilde{a}_k \cos\left(\frac{2\pi k}{KT_0} t\right)$$

with finite support:

$$| t | \leq \frac{KT_0}{2}$$

and:

$$g(t) = 0 \text{ outside support.}$$

For the classical PHYDYAS prototype with overlap factor:

$$K = 4$$

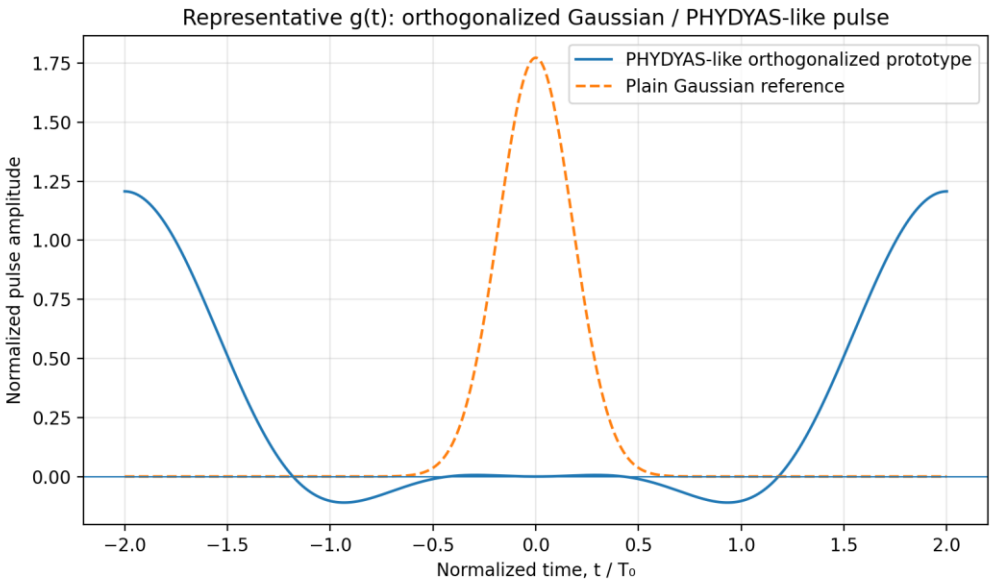
the coefficients are approximately:

$$a_0 = 1, a_1 = 0.97195983, a_2 = \frac{1}{\sqrt{2}}, a_3 = 0.23514695$$

So explicitly:

$$g(t) = 1 + 2a_1 \cos\left(\frac{2\pi t}{4T_0}\right) - 2a_2 \cos\left(\frac{4\pi t}{4T_0}\right) + 2a_3 \cos\left(\frac{6\pi t}{4T_0}\right)$$

after normalization.



The original Gaussian is:

$$g_G(t) = e^{-\pi(t/\sigma)^2}$$

which has optimal localization but not exact lattice orthogonality.

DIFFERENCE BETWEEN TWO USER EQUIPMENT (UE)

CURRENT UE (OFDM BASED)

UE-OFDM

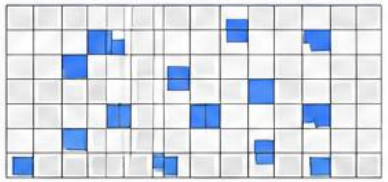
Focus: Throughput & Coverage
Doppler is an impairment to be compensated

Bits → FEC → QAM → OFDM (IFFT + CP) → DAC → RF Front End

RF Front End → ADC → OFDM (FFT) → Equalizer → QAM Demap → FEC Decode → Bits

SIGNAL PROCESSING VIEW

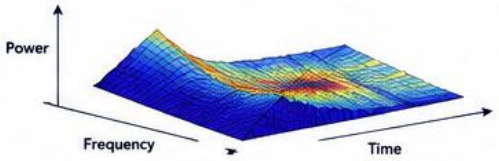
Time–Frequency Grid (Resource Elements)



Characteristics
- Subcarriers orthogonal in frequency
- Time slots orthogonal in time
- Doppler causes ICI
- Needs CP and channel estimation
- Mobility compensation required

MOBILITY & CHANNEL VIEW

Channel Effect in High Mobility



Impact
- High Doppler → severe ICI
- Performance drops
- Needs more pilots
- Handover harder

CAPABILITIES OF UE-OFDM

High Throughput	Mature Ecosystem	Limited Mobility Robustness	No Native Sensing	Lower Processing Demand
Good spectral efficiency in low mobility	Proven, widely deployed, cost optimized	Doppler is an impairment	Not designed for CNS (Communication, Navigation, Sensing)	FFT based, less complex

SUMMARY – UE-OFDM
- Designed for spectral efficiency in relatively low mobility.
- Doppler is a problem to be mitigated.

VS

FUTURE UE (OTFS/DD BASED)

UE-OTFS

Focus: Mobility, DD Awareness & CNS
Doppler is a native dimension (resource) to be used

BFN IC

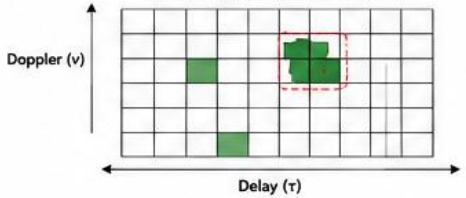
Bits → FEC → QAM → DD Mapper (Delay–Doppler) → OTFS IZak with pulse shaping → DAC → RF Front End

RF Front End → ADC → OTFS Zak with matched filter → DD Equalizer → QAM Demap → FEC Decode → Bits

OTFS Modem IC

SIGNAL PROCESSING VIEW

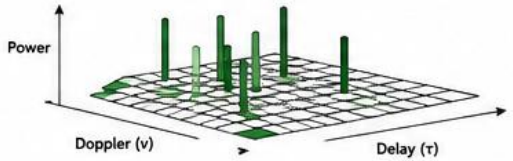
Delay–Doppler Grid (DD Bins)



Characteristics
- Resources in Delay–Doppler domain
- Doppler handled as a dimension
- Excellent high mobility performance
- Less sensitive to Doppler spread
- Sparse channel in DD domain

MOBILITY & CHANNEL VIEW

Channel Representation in DD Domain



Impact
- Channel is sparse in DD
- Robust to high Doppler
- Fewer pilots needed
- Smoother handover

CAPABILITIES OF UE-OTFS/DD

High Mobility Robustness	DD Self-Localization	CNS Support	DD-aware Beam Tracking	Higher Processing Capability
Doppler as a native dimension	UE can estimate its own delay/Doppler (position/velocity)	Enables Communication, Navigation and Sensing	Better beam selection and handover	New modem, more powerful

SUMMARY – UE-OTFS/DD
- Designed for high mobility, DD domain operation and CNS.
- Doppler is a dimension to be used, not an impairment.

USER MOBILE Handset - OTFS Transmitter ICs Example

Bits
↓
FEC Encoder
↓
QAM Mapper
↓
DD Resource Allocator
↓
DD Grid X[m,n]
↓
IZak Processor
↓
Pulse Shaping g(t)
↓
Digital Predistortion (optional)
↓
DAC
↓
Reconstruction Filter
↓
IQ Upconverter
↓
Driver Amplifier
↓
HPA

1. FEC Engine
Input:
`Bits`
Output:
`Coded bits`

Hardware:
•LDPC ASIC
•Polar ASIC
•BCH/RS (if desired)
Can be reused from OFDM systems.

2. QAM Mapper
Input:
`6 bits`
Output:

$$c_k$$

Example:

$$101001 \rightarrow 5 + 3j$$

Hardware:
•small lookup table
•trivial logic
Can be reused.

3. DD Resource Allocator
This is OTFS-specific.
Input:
`User symbols`
Output:

$$X[m, n]$$

Hardware:
`Scheduler`
`Memory Controller`
`DD Bin Allocation Table`
This is one of the new OTFS ASIC functions.

4. DD Memory
Stores:

$$X[m, n]$$

Example:
`256 × 256 complex symbols`

Hardware:
`SRAM`
`DDR`
`HBM`

5. IZak Processor
This is the heart of native OTFS.
Input:

$$X[m, n]$$

Output:

$$s[a]$$

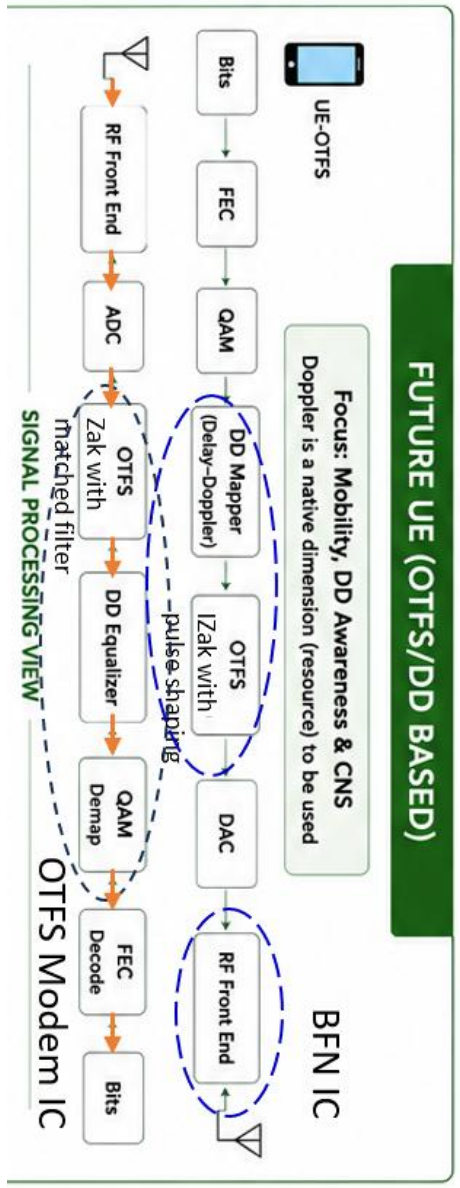
Hardware internally may use:
`FFT Engines`
`Memory Transpose`
`Phase Rotators`
even though architecturally it is IZak.
This is the second major OTFS-specific ASIC.

6. Pulse Shaping Engine
Implements:

$$g(t)$$

Examples:
•Gaussian
•Orthogonalized Gaussian
•IOTA

Hardware:
`FIR Filter`
`Polyphase Filter Bank`



Bits
↓
FEC Encoder
↓
QAM Mapper
↓
DD Resource Allocator
↓
DD Grid X[m,n]
↓
IZak Processor
↓
Pulse Shaping g(t)
↓
Digital Predistortion (optional)
↓
DAC
↓
Reconstruction Filter
↓
IQ Upconverter
↓
Driver Amplifier
↓
HPA

OTFS Transmitter

7. DPD (optional)
For GaN HPA.

Hardware:
`LUT`
`Adaptive DSP`
Can be reused from OFDM.

8. DAC
Input:

$$s[a]$$

Output:
analog waveform.

Hardware:
`High-speed DAC`
Can be reused.

9. Reconstruction Filter
Removes DAC images.
Hardware:
`LPF / BPF`

10. IQ Upconverter
Hardware:
`LO`
`IQ Mixer`
Can be reused.

11. Driver + HPA
Hardware:
`Driver Amp`
`GaN HPA`
Can be reused.

Illustrations

1. IZAK, ZAK computation example in communication system

Transmission: speech→PCM bits→4bit 16QAM labels→DD memory→XDD(τm,vn)→IZak→I[n],Q[n]→RF.

Live-speech OTFS transmission using 3GPP NR 16QAM

Assume a 4×4 DD frame:

$$M = 4, N = 4,$$

so the frame contains

$$MN = 16$$

DD bins.

Each 16QAM symbol represents four coded bits. Therefore,

$$16 \times 4 = 64 \text{ bits/frame.}$$

With 8-bit PCM, one frame carries eight speech samples.

1. Speech samples and PCM words

Take

$$s[n] = [-, 0.72, -, 0.18, 0.31, 0.77, 0.52, -0.410.08 - 0.93].$$

Using unsigned 8-bit PCM,

$$c[n] = \text{round}\left(\frac{s[n]+1}{2}\,255\right),$$

giving

$$c[n] = [36, 105, 167, 226, 194, 75, 138, 9].$$

Their binary words are

$$\begin{aligned} 36 &\rightarrow 00100100, \\ 105 &\rightarrow 01101001, \\ 167 &\rightarrow 10100111, \\ 226 &\rightarrow 11100010, \\ 194 &\rightarrow 11000010, \\ 75 &\rightarrow 01001011, \\ 138 &\rightarrow 10001010, \\ 9 &\rightarrow 00001001. \end{aligned}$$

The 64-bit stream is divided into sixteen 4-bit labels:

$$\begin{aligned} &0010,\ 0100,\ 0110,\ 1001, \\ &1010,\ 0111,\ 1110,\ 0010, \\ &1100,\ 0010,\ 0100,\ 1011, \\ &1000,\ 1010,\ 0000,\ 1001. \end{aligned}$$

Each DD-memory bin stores one 4-bit label.

This is

Unmapped $[X_{\mathrm{DD}}(\tau_r, \nu_k)]$ matrix

IZAK, ZAK computation example

2. 3GPP NR 16QAM mapping

For the label

$$b_0 b_1 b_2 b_3,$$

the complex symbol is

$$X_{\mathrm{DD}} = \frac{1}{\sqrt{10}}\left[(1-2b_0)\big(2-(1-2b_2)\big) + j(1-2b_1)\big(2-(1-2b_3)\big)\right].$$

The bits b_0, b_2 determine I, while b_1, b_3 determine Q.

For example,

$$0010:\ b_0 = 0,\ b_1 = 0,\ b_2 = 1,\ b_3 = 0.$$

Therefore,

$$I = (1-2\cdot 0)[2-(1-2\cdot 1)] = 3,$$
$$Q = (1-2\cdot 0)[2-(1-2\cdot 0)] = 1.$$

Hence,

$$0010 \longrightarrow \frac{3+j}{\sqrt{10}} = 0.948683 + j0.316228.$$

Let

$$\alpha = \frac{1}{\sqrt{10}}.$$

With Doppler coordinates as rows and delay coordinates as columns, the DD frame becomes

$$\mathbf{X}_{\mathrm{DD}} = \alpha \begin{bmatrix} 3+j & 1-j & 3-j & -1+3j \\ -3+j & 3-3j & -3-j & 3+j \\ -1-j & 3+j & 1-j & -3+3j \\ -1+j & -3+j & 1+j & -1+3j \end{bmatrix}.$$

The element in row k, column r is

$$X_{\mathrm{DD}}(\tau_r, \nu_k).$$

IZAK, ZAK computation example

3. To calculate s

$$s(t) = \sum_{m=0}^{M-1} \sum_{q=0}^{N-1} \left[\frac{1}{N} \sum_{n=0}^{N-1} X_{\mathrm{DD}}(\tau_m, \nu_n) \exp\left(j \frac{2\pi nq}{N} \right) \right] g_{\mathrm{tx}}(t - mT_s - qT)$$

$$v_m[q]$$

Sampled transmitted waveform

Sample the continuous waveform at (Tsamp=Ts, i.e., D=1)

$$t = \ell T_s.$$

Define

$$s[\ell] \equiv s(\ell T_s).$$

Using $T = MT_s$,

$$s[\ell] = \sum_{m=0}^{M-1} \sum_{q=0}^{N-1} v_m[q]\, g_{\mathrm{tx}}([\ell - m - Mq]T_s)$$

Condition under which $s[m + Mq] = v_m[q]$

When

$$g_{\mathrm{tx}}(pT_s) = \delta[p].$$

Also assume:

1. $T = MT_s$;
2. perfect timing alignment;

No additional interpolation samples are inserted between samples spaced by T_s.

At the sample index

$$\ell = m + Mq,$$

all shifted pulses except the corresponding pulse are zero. Hence,

$$s[m + Mq] = v_m[q]$$

IZAK, ZAK computation example

Under these conditions

$$s[m+Mq] = \frac{1}{N}\sum_{n=0}^{N-1} X_{\mathrm{DD}}(\tau_m, \nu_n) e^{j2\pi nq/N}.$$

$v_m[q]$

For $M = N = 4$

$$s[m+4q] = \frac{1}{4}\sum_{n=0}^{3} X_{\mathrm{DD}}(\tau_m, \nu_n) e^{j2\pi nq/4}$$

with

$$m = 0,1,2,3, q = 0,1,2,3.$$

The serialization is:

m=0, 1, 2, 3

$q = 0$: $s[0] = v_0[0],\ s[1] = v_1[0],\ s[2] = v_2[0],\ s[3] = v_3[0],$

$q = 1$: $s[4] = v_0[1],\ s[5] = v_1[1],\ s[6] = v_2[1],\ s[7] = v_3[1],$

$q = 2$: $s[8] = v_0[2],\ s[9] = v_1[2],\ s[10] = v_2[2],\ s[11] = v_3[2],$

$q = 3$: $s[12] = v_0[3],\ s[13] = v_1[3],\ s[14] = v_2[3],\ s[15] = v_3[3].$

Calculated values of s[n]

$$s[0] = -0.158114 + j0.158114,$$
$$s[1] = 0.316228 - j0.158114,$$
$$s[2] = 0.158114 - j0.158114,$$
$$s[3] = -0.158114 + j0.790569,$$
$$s[4] = 0.316228,$$
$$s[5] = 0.158114 + j0.316228,$$
$$s[6] = 0.316228 - j0.316228,$$
$$s[7] = 0.316228 + j0.316228,$$
$$s[8] = 0.474342 - j0.158114,$$
$$s[9] = 0.316228 + j0.158114,$$
$$s[10] = 0.474342 - j0.158114,$$
$$s[11] = -0.474342 + j0.158114,$$
$$s[12] = 0.316228 + j0.316228,$$
$$s[13] = -0.474342 - j0.632456,$$
$$s[14] = j0.316228,$$
$$s[15] = -j0.316228.$$

IZAK, ZAK computation example

These are separated into

$$I[n] = \Re\{s[n]\}, Q[n] = \Im\{s[n]\}.$$

The hardware path is

$X_{\mathrm{DD}}(\tau, \nu) \rightarrow \mathrm{IZak} \rightarrow s[n] = I[n] + jQ[n] \rightarrow I\mathrm{-DAC}, Q\mathrm{-DAC} \rightarrow \mathrm{IQ\ upconverter}.$

received RF → $\boldsymbol{I/Q}$ ADC → $\boldsymbol{y[n]}$ → Zak → DD equalization → 16QAM demapping → PCM words → reconstructed speech.

4. Receiver Zak transform

After downconversion and ADC sampling, the receiver obtains

$$y[n] = y_I[n] + jy_Q[n].$$

The Zak transform is [when $g_{rx}^*(t)=\delta(t)$]

$$\boxed{Y_{\mathrm{DD}}(\tau_r, \nu_k) = \sum_{q=0}^{N-1} y\,[r + qM] e^{-j2\pi kq/N}.}$$

For $M = N = 4$,

$$Y_{\mathrm{DD}}(\tau_r, \nu_k) = \sum_{q=0}^{3} y\,[r + 4q] e^{-j2\pi kq/4}.$$

For the ideal channel-free case,

$$y[n] = s[n],$$

so

$$\boxed{Y_{\mathrm{DD}}(\tau_r, \nu_k) = X_{\mathrm{DD}}(\tau_r, \nu_k).}$$

With an NTN channel,

$$Y_{\mathrm{DD}} \rightarrow \mathrm{DD-domain\ equalization} \rightarrow \hat{X}_{\mathrm{DD}}.$$

5. 16QAM demapping

Each equalized DD-bin value is assigned to the nearest allowed 3GPP 16QAM point.

For example,

$$\frac{3+j}{\sqrt{10}} \longrightarrow 0010.$$

Reading the bins in the agreed order recovers the original labels and therefore the original PCM integers:

$$\hat{c}[n] = [36, 105, 167, 226, 194, 75, 138, 9].$$

Thus,

$$\boxed{\hat{c}[n] = c[n].}$$

IZAK, ZAK computation example

6. Speech reconstruction

The reconstructed normalized speech samples are

$$\hat{s}[n] = 2\frac{\hat{c}[n]}{255} - 1.$$

Hence,

$\hat{s}[n]$

$= [-, 0.717647, -, 0.176471, 0.309804, 0.772549, 0.521569, -0.4117650.082353 - 0.929412].$

The PCM words are recovered exactly. The small difference from the original amplitudes is only due to the initial 8-bit quantization.

The complete chain is

At the Transmitter,

speech → PCM bits → 4−bit 16QAM labels → DD memory → 3GPP mapper → $X_{\mathrm{DD}}(\tau, \nu)$ → IZak → $I[n], Q[n]$ → RF transmission.

At the receiver,

received RF → I/Q ADC → $y[n]$ → Zak → DD equalization → 16QAM demapping → PCM words → reconstructed speech.

OTFS frame duration

One OTFS period:

$$T = \frac{M}{B}$$

Total frame duration:

$$T_{\mathrm{frame}} = NT = \frac{MN}{B}$$

2. Channel estimation in DD domain

Get H_{DD} by **sending known DD reference symbols** and measuring how they appear in the received DD grid.
The model is:

$$Y = H_{\mathrm{DD}}X + W$$

where

$$X = \text{transmitted DD symbol vector}$$
$$Y = \text{received DD symbol vector after Zak projection}$$
$$H_{\mathrm{DD}} = \text{effective DD channel matrix}$$

Direct pilot method
Transmit a known DD pilot:

$$X_p[m, n]$$

Receive:

$$Y_p[m, n]$$

Then estimate:

$$\boxed{\widehat{H}_{\mathrm{DD}} = Y_p X_p^{\dagger}}$$

where: $X_p^{\dagger}$ is the pseudo-inverse of the known pilot matrix.

Also used for MCS decision

Control Anchor
+
Pilot DD Bins
↓
Estimate H_DD
↓
Compute post-equalization SINR
↓
Choose Mod. and Coding scheme (MCS)
↓
Allocate DD payload bins

Channel estimation in DD domain

Assume the DD-domain pilot relation

$$\boxed{\mathbf{Y}_{\mathrm{DD}} = \mathbf{H}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}} + \mathbf{W}_{\mathrm{DD}},}$$

where

$$\mathbf{X}_{\mathrm{DD}}, \mathbf{Y}_{\mathrm{DD}} \in \mathbb{C}^{M\times N}, \mathbf{H}_{\mathrm{DD}} \in \mathbb{C}^{M\times M}.$$

The objective is to design $\mathbf{X}_{\mathrm{DD}}$so that $\mathbf{H}_{\mathrm{DD}}$is obtained with minimum computation.

1. Design an orthogonal QPSK pilot matrix

For $M = N = 4$, choose

$$p = \frac{1+j}{\sqrt{2}}, \mid p \mid = 1,$$

and

$$\boxed{\mathbf{X}_{\mathrm{DD}} = p\begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}.}$$

Thus, every element is either

$$\frac{1+j}{\sqrt{2}}$$

or

$$\frac{-1-j}{\sqrt{2}},$$

both valid unit-magnitude QPSK symbols.

Define

$$\mathbf{A} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{bmatrix}.$$

The rows are orthogonal:

$$\mathbf{A}\mathbf{A}^{H} = 4\mathbf{I}_4.$$

Therefore,

$$\boxed{\mathbf{X}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}}^{H} = 4\mathbf{I}_4.}$$

Hence,

$$\boxed{\mathbf{X}_{\mathrm{DD}}^{\dagger} = \frac{1}{4}\mathbf{X}_{\mathrm{DD}}^{H}.}$$

Channel estimation in DD domain

2. Receive the pilot DD matrix

After IZak transmission, propagation and Zak reception, the satellite obtains the known 4×4 received matrix

$$\mathbf{Y}_{\mathrm{DD}}.$$

3. Calculate the channel matrix

From

$$\mathbf{Y}_{\mathrm{DD}} = \mathbf{H}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}} + \mathbf{W}_{\mathrm{DD}},$$

the least-squares channel estimate is

$$\boxed{\widehat{\mathbf{H}}_{\mathrm{DD}} = \mathbf{Y}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}}^{\dagger}.}$$

Using the orthogonal pilot property,

$$\boxed{\widehat{\mathbf{H}}_{\mathrm{DD}} = \frac{1}{4}\mathbf{Y}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}}^{H}.}$$

Since

$$\mathbf{X}_{\mathrm{DD}}^{H} = \frac{1-j}{\sqrt{2}}\mathbf{A},$$

the explicit calculation is

$$\boxed{\widehat{\mathbf{H}}_{\mathrm{DD}} = \frac{1-j}{4\sqrt{2}}\,\mathbf{Y}_{\mathrm{DD}}\begin{bmatrix}1 & 1 & 1 & 1\\ 1 & -1 & 1 & -1\\ 1 & 1 & -1 & -1\\ 1 & -1 & -1 & 1\end{bmatrix}.}$$

This single matrix multiplication gives the complete estimated DD channel matrix.

In the absence of noise,

$$\boxed{\widehat{\mathbf{H}}_{\mathrm{DD}} = \mathbf{H}_{\mathrm{DD}}.}$$

With noise,

$$\boxed{\widehat{\mathbf{H}}_{\mathrm{DD}} = \mathbf{H}_{\mathrm{DD}} + \frac{1}{4}\mathbf{W}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}}^{H}.}$$

Channel estimation in DD domain

4. General $M \times N$ pilot

For $N \geq M$, design the QPSK pilot matrix such that

$$\boxed{\mathbf{X}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}}^{H} = N\mathbf{I}_{M}.}$$

Then

$$\boxed{\mathbf{X}_{\mathrm{DD}}^{\dagger} = \frac{1}{N}\mathbf{X}_{\mathrm{DD}}^{H}}$$

and the channel matrix is obtained directly as

$$\boxed{\widehat{\mathbf{H}}_{\mathrm{DD}} = \frac{1}{N}\mathbf{Y}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}}^{H}.}$$

For $M = N = 128$,

$$\boxed{\widehat{\mathbf{H}}_{\mathrm{DD}} = \frac{1}{128}\mathbf{Y}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}}^{H}.}$$

Therefore, the complete minimum-operation procedure is

$$\boxed{\text{orthogonal QPSK } \mathbf{X}_{\mathrm{DD}} \rightarrow \text{measure } \mathbf{Y}_{\mathrm{DD}} \rightarrow \widehat{\mathbf{H}}_{\mathrm{DD}} = \frac{1}{N}\mathbf{Y}_{\mathrm{DD}}\mathbf{X}_{\mathrm{DD}}^{H}.}$$

5. Fractional DD offset

Physical channel may land at m+Δm, n+Δn.

Fractional offsets create leakage and phase ramps.

Handle in H_DD, equalizer, or fine pre-compensation.

$$Y = H_{\mathrm{eff}}(\Delta m, \Delta n)X + W$$

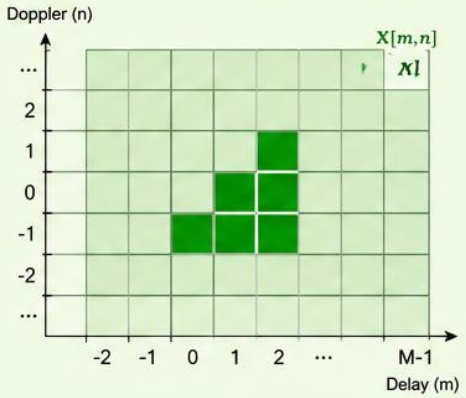
$$\hat{X} = \left(H_{\mathrm{eff}}^{H}H_{\mathrm{eff}} + \sigma^{2}I\right)^{-1}H_{\mathrm{eff}}^{H}Y$$

This is where Δm, Δn matter

3. Real world user doesn't always land on integer dd grid – Handling fractional offset for dd bins

TRANSMITTER (User)

1. Allocated DD Resources (Integer Bins)



2. IZak Transform (DD → Time)

$s(t)$

3. Transmit Waveform



CHANNEL (Propagation)

Physical Channel Impacts

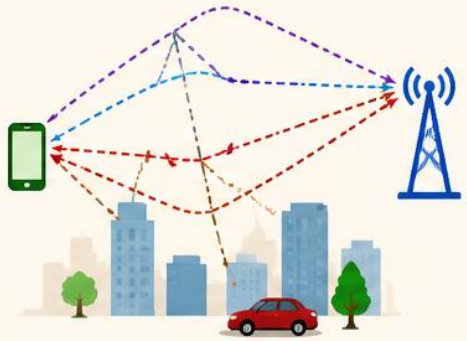
Delay	Doppler
$\tau = \tau_i + \Delta\tau$	$\nu = \nu_i + \Delta\nu$
τ_i : integer delay bin	ν_i : integer Doppler bin
$\Delta\tau$: fractional delay	$\Delta\nu$: fractional Doppler

Multipath Channel

$$h(t,\tau,\nu) = \sum_{l=1}^{L} \alpha_l \delta(t-\tau_l) e^{j2\pi\nu_l t}$$

Received Signal

$$r(t) = h(t,\tau,\nu) * s(t) + w(t)$$



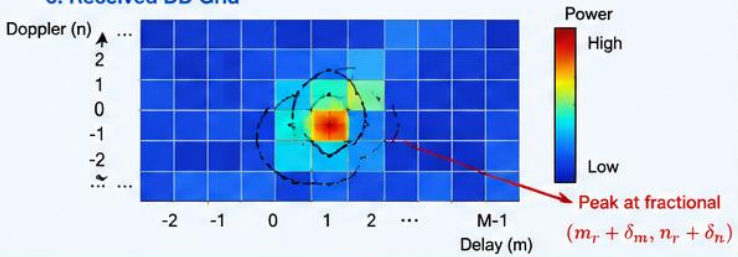
RECEIVER (Base Station)

4. Received Waveform $r(t)$



5. Zak Transform (Time → DD) $\mathbf{Y}[m,n]$

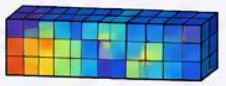
6. Received DD Grid



7. Effective DD Channel Model

$$\mathbf{Y} = \mathbf{H}_{eff}\mathbf{X} + \mathbf{W}$$

$\mathbf{H}_{eff} =$



$\mathbf{H}_{eff}$ includes:

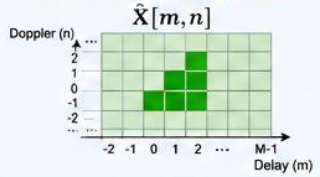
- Physical channel
- Integer DD shift
- Fractional delay leakage
- Fractional Doppler leakage
- Pulse-shaping effects

8. DD Equalization (Combined Inversion)

$$\hat{\mathbf{X}} = \left(\mathbf{H}_{eff}^H \mathbf{H}_{eff} + \sigma^2 \mathbf{I}\right)^{-1} \mathbf{H}_{eff}^H \mathbf{Y}$$

(MMSE Equalizer)

9. Recovered DD Symbols $\hat{\mathbf{X}}[m,n]$



Key Insight

Even if user transmits on integer DD bins, channel causes fractional $(\Delta\tau, \Delta\nu)$ → energy spreads around fractional DD location.

Unified View

Fractional DD effects are NOT separately corrected first. They are embedded in H_{eff} and equalized together.

Mathematically

$$\mathbf{H}_{eff} = \mathbf{H}_{channel} \cdot \mathbf{H}_{int\ shift} \cdot \mathbf{H}_{frac\ delay} \cdot \mathbf{H}_{frac\ Doppler} \cdot \mathbf{H}_{pulse}$$

$$\hat{\mathbf{X}} = \mathbf{H}_{eff}^{-1}\mathbf{Y} \quad (\text{or} \quad \left(\mathbf{H}_{eff}^H \mathbf{H}_{eff} + \sigma^2 \mathbf{I}\right)^{-1} \mathbf{H}_{eff}^H \mathbf{Y})$$

- Integer DD assignment → scheduling
- Fractional DD → tracking & channel
- Equalization → one-shot inversion

IZak Transform with fractional dd offset compensation - computation load over normal IZAK

$$\Delta\tau = \Delta m\, T_s, \qquad \Delta\nu = \frac{\Delta n}{NT}, \qquad T = MT_s.$$

For the physical fractional DD shift with operator order

$$\mathcal{M}_{\Delta\nu}\mathcal{T}_{\Delta\tau},$$

the formula is

$$s_{\Delta m,\Delta n}(t) = \sum_{m=0}^{M-1} \sum_{q=0}^{N-1} B_{\Delta m,\Delta n}[m, q] \times g_{\Delta\nu}(t - (m + \Delta m)T_s - qT),$$

Where, $g_{\Delta\nu}(u) = e^{j2\pi\Delta\nu u} g_{\mathrm{tx}}(u)$

And $B_{\Delta m,\Delta n}[m, q] = v_m[q]\, e^{j2\pi\Delta nq/N} e^{j2\pi\Delta n(m+\Delta m)/(MN)}$

With the ordinary IZak coefficient

$$v_m[q] = \frac{1}{N}\sum_{n=0}^{N-1} X[m, n] e^{j2\pi nq/N}$$

Thus,

$$B_{\Delta m,\Delta n}[m, q] = e^{j2\pi\Delta nq/N} e^{j2\pi\Delta n(m+\Delta m)/(MN)} \times \frac{1}{N}\sum_{n=0}^{N-1} X[m, n] e^{j2\pi nq/N}.$$

For computation-load counting, the two outer phase terms should be combined:

$$P_{\Delta m,\Delta n}[m, q] = \exp\left[j2\pi\Delta n \frac{qM + m + \Delta m}{MN}\right],$$

so that

$$B_{\Delta m,\Delta n}[m, q] = P_{\Delta m,\Delta n}[m, q]\, v_m[q]$$

Hence the incremental coefficient-processing load beyond ordinary IZak is one complex multiplication for each processed (m, q):

$$\boldsymbol{C_{B,\mathrm{extra}}} = \boldsymbol{MN} \text{ complex multiplications per full frame} +$$

4. Bandwidth incompatibility among Satellite, BS and UE

Do **not** transmit a full 500 MHz satellite waveform to a 100 MHz UE and expect recovery.

That would cause:

UE RF filter → loss of 400 MHz signal content
→ irrecoverable DD distortion.

Do **not** allocate arbitrary 500 MHz DD bins to a 100 MHz user.

Instead, **the scheduler must allocate only DD regions belonging to the UE's 100 MHz-compatible tile group.**

The satellite may have a 500 MHz DD frame:

$$Y_{\text{sat}}[m, n], m = 0, \dots, M_{\text{sat}} - 1.$$

But resources are grouped into bandwidth-compatible DD regions.

For example:

$$\mathcal{B}_{\text{sat}} = \mathcal{B}_0^{(1)} \cup \mathcal{B}_0^{(2)} \cup \cdots \cup \mathcal{B}_0^{(10)},$$

where each $\mathcal{B}_0^{(i)}$ is a 50 MHz-equivalent tile.

A UE allocation uses two such tiles:

$$\mathcal{B}_{\text{UE}} = \mathcal{B}_0^{(i)} \cup \mathcal{B}_0^{(i+1)}.$$

A BS allocation uses five tiles:

$$\mathcal{B}_{\text{BS}} = \bigcup_{i=1}^{5} \mathcal{B}_0^{(i)}.$$

A full satellite beam uses all ten tiles.

Therefore the DD memory-remap rule becomes:

$$(\text{input beam}, \text{input tile group}, m, n) \rightarrow (\text{output beam}, \text{output tile group}, m', n').$$

The switching is still DD-memory remap, but now with a bandwidth/tile hierarchy.

This matches the native DD switch idea while respecting the different RF front-end bandwidths.

Routing Descriptor

{source beam, source tile group, DD block start, DD block size, destination beam, destination tile group, destination user/BS}.

5. Clock and synchronization requirement

For FFT-accelerated IZak, the total number of radix-2 butterflies per frame is

$$M\frac{N}{2}\log_2 N = 128 \times 64 \times 7 = 57{,}344.$$

At the frame rate

$$30{,}517.6 \text{ frames/s},$$

this is

$$57{,}344 \times 30{,}517.6 \approx 1.75 \times 10^9$$

complex butterflies per second per beam.

With P-way parallel output, the average number of butterflies active per clock is

$$\frac{1.75 \times 10^9}{500 \times 10^6 / P} = 3.5P.$$

So:

P	IZak clock	Average complex butterflies per clock
1	500 MHz	3.5
4	125 MHz	14
8	62.5 MHz	28
16	31.25 MHz	56
32	15.625 MHz	112
64	7.8125 MHz	224
128	3.90625 MHz	448

Clock and synchronization requirement

IZAK-ZAK computation clock can be low. Clock specs are set by ADC and DAC, I/Q DAC sampling clock ≈ 1 GHz each
For each BS, satellite payload and gateway:

GNSS / network time reference $\rightarrow$ 10 MHz / 100 MHz reference $\rightarrow$ PLL / synthesizer $\rightarrow$ ADC, DAC, LO clocks

The system does **not** require perfect transmitter–receiver clock locking. It requires:

$$\boxed{\begin{array}{c}\text{disciplined independent clocks} + \text{pilot}-\text{based frame acquisition} + \\ \text{CFO/Doppler de}-\text{rotation} + \text{fractional}-\text{delay tracking} + \\ \text{DD channel}-\text{matrix correction.}\end{array}}$$

Numerically, for the current design:

$$\boxed{\text{coarse timing accuracy: } 150\text{–}200 \text{ ns}}$$

$$\boxed{\text{residual common CFO after correction: } 500\text{–}1000 \text{ Hz}}$$

$$\boxed{\text{DAC clock jitter target: } \lesssim 1 \text{ ps RMS}}$$

$$\boxed{\text{timing drift at } 8000 \text{ m/s: } 26.7 \text{ ns/ms.}}$$

That is a realistic synchronization plan: loose global clock discipline, precise local pilot tracking, and residual correction through the measured DD channel matrix.